Hyrql characterization of functions computable in quantum polynomial time

Kostia Chardonnet, Emmanuel Hainry, Romain Péchoux, Thomas Vinet

arxiv:2510.20452v2; CC BY4.0

Editorial note. Fixed arXivv2 independently carries CC BY4.0; HAL05518918v1 also identifies CC BY4.0 and links the same v2 filename. Native full proof appendix is retained.

Editorial note. All active native TeX inputs mechanically flattened with their original comments intact, preserving percent-based line joining and author mathematics. Publisher metadata is replaced by article wrapper; shared author theorem environments supplied.

Editorial note. Only the selected characterization is counted. Both directions of the quantum polynomial-time characterization count together; standard typing support and compilation lemmas stay uncounted.

Editorial scope. Complete author material collection for one structural theorem; author retains responsibility for the mathematics.

Allowed standard prerequisites (editorial).

- Typed PCF universal halting completeness, algebraic-number effective operations, and uniform quantum computation models. Original author references in bibli.bib and exact assumptions retained; no recursive reproving of standard prior theorems.

1 Introduction

Motivations. Most well-known quantum algorithms, such as Shor's algorithm [48], were historically designed on the Quantum Random Access Machine (QRAM) model [33]. In this model, a program interacts with a quantum memory through basic operations complying with the laws of quantum mechanics. These operations include a fixed set of quantum gates, chosen to be universal, as well as a probabilistic measurement of qubits. Consequently, the control flow is purely classical, i.e., depends on the (classical) outcome of a measure. Based on this paradigm, several high-level quantum programming languages have been introduced, each with different purposes and applications, from assembly code [18, 19], to imperative languages [26], circuit description languages [28], and λ-calculi [47].

A relevant issue was to extend these models to programs with *quantum control*, also known as coherent control, enabling a "program as data" treatment

for the quantum paradigm. Quantum control consists in the ability to write superposition of programs in addition to superposition of data and increases the expressive power of quantum programming languages. It allows the programmer to write algorithms such as the *quantum switch*, which uses fewer resources (quantum gates) than algorithms with classical control [16] and is physically implementable [1, 43]. Hence, quantum control provides a computational advantage over classical control [3, 49, 34]. In the last decades, several quantum programming languages implementing this concept have been designed, non-exhaustively [2, 44, 24, 37, 56]. To get the best of both worlds, a natural next step was the development of *hybrid languages*, i.e., languages that allow classical and quantum flow/data to be combined. The hybrid paradigm is conveying considerable practical interest: for example, it is used by quantum variational algorithms, a class of quantum algorithms leveraging both classical and quantum computing resources to find approximate solutions to optimization problems; and the properties of hybrid languages have also been deeply studied, non-exhaustively [51, 54, 21, 9].

Since hybrid languages offer interesting prospects in terms of expressiveness and optimal resource consumption, a relevant and open issue concerns the development of resource-aware (static) analyses of their programs. These static analyses can be applied to predict the low-level resources required to execute quantum algorithms, such as the depth or size of a quantum circuit.

Contributions. This work solves the above issue for the first time by introducing a HYbrid Recursive Quantum Language `Hyrql` on which a resource analysis can be performed. `Hyrql` is a hybrid extension of the functional quantum language `Spm` (*Symmetrical pattern-matching*) of [44, 37]. In `Spm`, the programmer can write down directly unitary applications, combining quantum superposition of terms with pattern-matching. Unitarity is enforced by a (decidable) linear typing discipline using an orthogonality predicate. `Spm` is, however, a purely quantum language, which makes the manipulation (and, consequently, resource analysis) of classical information awkward: they can be neither discarded nor duplicated by linearity. The hybrid nature of `Hyrql` relies on a typing discipline which delineates a clear separation between quantum/linear and classical/non-linear data in the typing contexts. Similarly to `Spm`, `Hyrql` does not include measurement: the flow from quantum data to classical data is handled through the use of a `shape` construct, as introduced in [28], which returns the classic structural information on data containing quantum information, without getting any information on the value of the quantum states. For example, the `shape` of a qubit list provides classical (duplicable) information, such as the number of qubits. The decision to provide `Hyrql` with a `Spm`-architecture was motivated by two important factors. First, in contrast with most of its competitors, `Spm` does not require the inclusion of an initial set of quantum gates, making its resource analysis generic. Second, thanks to its pattern-matching design, `Spm` allows for the reuse of a wide variety of tools designed for resource analysis (e.g., termination or complexity) of term rewriting systems.

Our paper contains the following main contributions:

- The introduction of the hybrid language with general recursion `Hyrql`, as well as its operational semantics, type system, and standard properties: confluence (Theorem 11), progress (Theorem 13), and subject reduction (Theorem 14).

- A proof that the orthogonality predicate in `Hyrql` is Π_2^0-complete, hence undecidable (Theorem 16). Consequently, typing is not decidable. This is not surprising as the language encompasses general recursion. The decidability of typing can still be recovered, on expressive sub-fragments of the language, e.g., when `Hyrql` is restricted to finite types and terminating programs (Proposition 17).

- A compilation from terms (of the good type) to quantum circuits, whose size is bounded by the runtime-complexity of the initial term (Theorem 25). In particular, terms that terminate in polynomial time characterize exactly the complexity class `FBQP` (Theorem 27 and 28). This result is not trivial, as lambda calculi do not comply with van Emde Boas' invariant thesis [50], i.e., a polynomial time reduction on a lambda term may correspond to an exponential time reduction on a Turing machine.

- An overview on how complexity results can be obtained and automatized by translating our language into term-rewrite systems, to use existing techniques for runtime analysis (e.g., interpretations [38], recursive path orderings [22, 23], dependency pairs [4], size-change principle [36]).

We illustrate the expressive power of the language through several examples: implementation of basic quantum gates (Hadamard gate, Example 1), higher-order (quantum switch, Example 2), general recursion (bounded quantum walk, Example 3), and hybrid control flow (protocol *BB84* [10], Example 30).

	`Hyrql`	`FOQ` [29]	`RQC`$^{++}$ [55]	`Silq` [12]	`Qunity` [51]	`DLPZ` [21]
Resource-aware	✓	✓	✗	✗	✗	✗
General recursion	✓	✗	✓	✗	✗	✗
Higher-order	✓	✗	✗	✗	✗	✗

Table 1: Comparison table between hybrid quantum languages

Related works. In Table 1, we provide a non-exhaustive comparison between the main families of hybrid quantum languages which treat classical data as first-class citizen. Towards that end, we consider the three following criteria.

- *Resource-aware* highlights if the language is designed towards resource analysis, that is, allows to characterize well-known complex classes of quantum computational models. This is the case of `FOQ` [29, 30], which characterizes, under restrictions, quantum polynomial time and quantum polylogarithmic time [27]. However, `Hyrql` has a greater expressive power than `FOQ` as it is not restricted to unitary operators. Moreover, all standard inductive datatypes can be expressed in `Hyrql`, whereas `FOQ` is restricted to lists of qubits.

- *General recursion* specifies whether the language includes unbounded recursion on classical data. The ability to handle this type of recursion is necessary for a resource analysis to be relevant (i.e., not trivial as in the case of strong normalizing languages).

$$
\begin{array}{lll}
\text{(Values)} & v ::= & x \mid |0\rangle \mid |1\rangle \mid c(v_1, \ldots, v_n) \mid \lambda x.t \mid \texttt{letrec}\, f\, x = t \mid \sum_{i=1}^{n} \alpha_i \cdot v_i \\
\text{(Terms)} & t ::= & x \mid |0\rangle \mid |1\rangle \mid \texttt{qcase}\, t \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\} \\
& & \mid c(t_1, \ldots, t_n) \mid \texttt{match}\, t \left\{ c_1(\overrightarrow{x_1}) \to t_1 \,, \ldots, c_n(\overrightarrow{x_n}) \to t_n \right\} \\
& & \mid \lambda x.t \mid \texttt{letrec}\, f\, x = t \mid t_1 t_2 \mid \sum_{i=1}^{n} \alpha_i \cdot t_i \mid \texttt{shape}(t)
\end{array}
$$

Table 2: Syntax of the `Hyrql` language

- *Higher-order* denotes the ability of the language to feature general higher-order terms, i.e., functions taking functions as input. While some languages of Table 1 have syntactic constructs to do so, most of them are restricted to first-order calls.

As we are interested in resource analysis in the general setting, Table 1 only compares `Hyrql` with hybrid languages, rather than with existing languages with quantum control, non-exhaustively [44, 24, 54, 56]. Other studies have already been carried out on resource analysis in quantum programming languages. They use techniques (e.g., type systems) developed in the field of Implicit Computational Complexity (ICC, see [42]) to characterize quantum complexity classes [20], to infer the expected cost or expected value of a quantum program [6], or to infer upper bounds on quantum resources (depth and size of circuits) [17]. However, these studies are restricted to classical control. For example, [20] studies the use of soft linear logic [35] on a quantum lambda calculus [46], [6] adapts expectation transformers on a language based on `QPL` [45], and [17] develops a dependent type system on a variant of the Quipper circuit description language [28].

2 The `Hyrql` Programming Language

We introduce the syntax, the operational semantics, and the type system of `Hyrql` along with illustrating examples.

2.1 Syntax

The syntax of Terms and Values in `Hyrql` is provided in Table 2. Terms feature variables, noted $x, y, f, \ldots$ and taken from an infinite countable set of variables. We denote by $\overrightarrow{e}$ a (finite and possible empty) sequence of elements $e_1, \ldots, e_n$. Qubits are introduced through basis states $|0\rangle, |1\rangle$, allowing us to write quantum conditionals $\texttt{qcase}\, t \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\}$, which corresponds to the superposition of t_0 and t_1 controlled by qubit t. `Hyrql` also features classical constructors c^s consisting of a constructor symbol c, and a signature s indicating the type of the constructor. This will be formalized in Section 2.3 and will be used only when required. Constructors can be used in constructor application $c(t_1, \ldots, t_n)$ and through pattern-matching, sometimes abbreviated as $\texttt{match}_{1 \leq i \leq n}\, t \, \{c_i(\overrightarrow{x_i}) \to t_i\}$. Each constructor symbol c comes with a fixed arity and is always fully applied. We assume the existence of standard inductive constructors for unit $()$, tensor products $\otimes$, natural numbers 0 and S, and lists

$$t_1 + t_2 \equiv t_2 + t_1 \qquad t_1 + (t_2 + t_3) \equiv (t_1 + t_2) + t_3 \qquad 1 \cdot t \equiv t \qquad t + 0 \cdot t' \equiv t$$
$$\alpha \cdot (\beta \cdot t) \equiv (\alpha\beta) \cdot t \qquad \alpha \cdot (t_1 + t_2) \equiv \alpha \cdot t_1 + \alpha \cdot t_2 \qquad \alpha \cdot t + \beta \cdot t \equiv (\alpha + \beta) \cdot t$$

$$\texttt{qcase}\,(\textstyle\sum_{j=1}^{m} \alpha_j \cdot s_j)\,\{|0\rangle \to t_0\,, |1\rangle \to t_1\} \equiv \textstyle\sum_{j=1}^{m} \alpha_j \cdot (\texttt{qcase}\, s_j\, \{|0\rangle \to t_0\,, |1\rangle \to t_1\})$$
$$c(\overrightarrow{t_1}, \textstyle\sum_{j=1}^{m} \alpha_j \cdot s_j, \overrightarrow{t_2}) \equiv \textstyle\sum_{j=1}^{m} \alpha_j \cdot c(\overrightarrow{t_1}, s_j, \overrightarrow{t_2})$$
$$\texttt{match}_{1\le i\le n}\,(\textstyle\sum_{j=1}^{m} \alpha_j \cdot s_j)\,\{c_i(\overrightarrow{x_i}) \to t_i\} \equiv \textstyle\sum_{j=1}^{m} \alpha_j \cdot (\texttt{match}_{1\le i\le n}\, s_j\, \{c_i(\overrightarrow{x_i}) \to t_i\})$$
$$t(\textstyle\sum_{i=1}^{m} \alpha_j \cdot s_j) \equiv \textstyle\sum_{j=1}^{m} \alpha_j \cdot t\, s_j$$
$$C_{\equiv}[t] \equiv C_{\equiv}[t'] \qquad \text{when } t \equiv t'$$

Table 3: Equivalence relation $\equiv \subseteq$ Terms $\times$ Terms

[ ] and ::. For convenience, constructors are sometimes used in an infix notation. For example, tensor products will be written as $x \otimes y$ and list constructors as $h :: t$.

Higher-order is featured via a standard λ-abstraction $\lambda x.t$, and a construct for general recursion $\texttt{letrec}\, f\, x = t$. Term application is denoted by $t_1 t_2$.

Given amplitudes $\alpha_i \in \mathbb{C}$, the term $\sum_{i=1}^{n} \alpha_i \cdot t_i$ represents a *superposition* of terms t_i. In the special case where $n = 2$, we just write $\alpha_1 \cdot t_1 + \alpha_2 \cdot t_2$. In a dual manner, the `shape` construct, introduced in [28], returns the classic structural information on data containing quantum information. For example, the shape of a qubit list is a copy of the list structure without the qubits (see Example 6).

In order to avoid conflicts between free and bound variables, we will always work up to α-conversion and use Barendregt's convention [8, p. 26] which consists in keeping all bound and free variable names distinct, even when this remains implicit. We also define a partial order $\lhd$ on terms, where $s \lhd t$ if s is a *strict subterm* of t. We denote $\trianglelefteq$ as the reflexive closure of $\lhd$. The *size of a term* is written $|t|$ and is defined standardly in Definition 29.

As we work with amplitudes and superpositions, Terms and Values have to be considered with respect to a vector space structure. Towards that end, we define an equivalence relation $\equiv$ in Table 3. Such definition relies on *equivalence contexts*, defined by the following grammar:

$$\begin{aligned}\text{(Equiv. contexts)}\quad C_{\equiv} ::= \diamond \mid \texttt{qcase}\, C_{\equiv}\, \{|0\rangle \to t_0\,, |1\rangle \to t_1\} \mid c(\overrightarrow{t_1}, C_{\equiv}, \overrightarrow{t_2}) \mid tC_{\equiv} \mid C_{\equiv}t \\ \mid \texttt{match}_{1\le i\le n}\, C_{\equiv}\, \{c_i(\overrightarrow{x_i}) \to t_i\} \mid t + C_{\equiv} \mid \texttt{shape}(C_{\equiv})\end{aligned}$$

An equivalence context $C_{\equiv}$ is thus a term with one hole $\diamond$; let $C_{\equiv}[t]$ be the term obtained by filling the hole with t in $C_{\equiv}$. The first rules of Table 3 make the definition of the $\sum$ construct unambiguous/sound, i.e., it corresponds exactly to repeated applications of the $+$ construct. The last rules of Table 3 are commutation rules highlighting the linearity of quantum terms.

Example 1 (Quantum State and Hadamard Gate)**.** *We can define the orthogonal basis states as* $|\pm\rangle \triangleq \frac{1}{\sqrt{2}} \cdot |0\rangle \pm \frac{1}{\sqrt{2}} \cdot |1\rangle$*. The Hadamard gate can be encoded in* `Hyrql`*:*

$$\texttt{Had} \triangleq \lambda x.\texttt{qcase}\, x\, \{|0\rangle \to |+\rangle\,, |1\rangle \to |-\rangle\}$$

$$\frac{}{\texttt{qcase}\ |0\rangle\ \{|0\rangle \to t_0\,, |1\rangle \to t_1\} \rightsquigarrow t_0}\ (\text{Qcase}_0) \qquad \frac{}{\texttt{qcase}\ |1\rangle\ \{|0\rangle \to t_0\,, |1\rangle \to t_1\} \rightsquigarrow t_1}\ (\text{Qcase}_1)$$

$$\frac{}{\texttt{match}_{1\le i\le n}\, c_j(\overrightarrow{v_p})\ \{c_i(\overrightarrow{x_i}) \to t_i\} \rightsquigarrow t_j\{\overrightarrow{v_p}/\overrightarrow{x_j}\}}\ (\text{Match}) \qquad \frac{}{(\lambda x.t)v_p \rightsquigarrow t\{v_p/x\}}\ (\text{Lbd})$$

$$\frac{}{(\texttt{letrec}\, f\, x = t)v_p \rightsquigarrow t\{\texttt{letrec}\, f\, x = t/f, v_p/x\}}\ (\text{Rec}) \qquad \frac{\sum_{i=1}^n \alpha_i \cdot p_i \in \texttt{CAN} \setminus \text{Value} \qquad p_i \rightsquigarrow^? t_i}{\sum_{i=1}^n \alpha_i \cdot p_i \rightsquigarrow \sum_{i=1}^n \alpha_i \cdot t_i}\ (\text{Can})$$

$$\frac{}{\texttt{shape}(|0\rangle) \rightsquigarrow ()}\ (\text{Shape}_0) \qquad \frac{}{\texttt{shape}(|1\rangle) \rightsquigarrow ()}\ (\text{Shape}_1) \qquad \frac{}{\texttt{shape}(c^s(\overrightarrow{v_p})) \rightsquigarrow c^{\texttt{shape}(s)}(\texttt{shape}(\overrightarrow{v_p}))}\ (\text{Shape}_c)$$

$$\frac{\sum_{i=1}^n \alpha_i \cdot v_i \in \texttt{CAN}}{\texttt{shape}(\sum_{i=1}^n \alpha_i \cdot v_i) \rightsquigarrow \texttt{shape}(v_1)}\ (\text{Shape}_s) \qquad \frac{t \rightsquigarrow t'}{\texttt{shape}(t) \rightsquigarrow \texttt{shape}(t')}\ (\text{Shape}_E)$$

$$\frac{p \rightsquigarrow t}{E[p] \rightsquigarrow E[t]}\ (E) \qquad \frac{t \equiv t_1 \qquad t_1 \rightsquigarrow t_1' \qquad t_1' \equiv t'}{t \rightsquigarrow t'}\ (\text{Equiv})$$

Table 4: Reduction rules of the language

Example 2 (Quantum Switch)**.** *The quantum switch [16] is a program which, on two unitaries U, V and a two-qubit state $|\phi\rangle$, applies UV and VU, in superposition, to the second qubit, controlled by the value of the first qubit. This can be implemented in* `Hyrql` *by* `QS` $U\ V\ |\phi\rangle$*, where* `QS` *is defined as:*

$$\texttt{QS} \triangleq \lambda f.\lambda g.\lambda q.\texttt{match}\, q\, \{c \otimes t \to \texttt{qcase}\, c\, \{|0\rangle \to |0\rangle \otimes f(g\, t)\,, |1\rangle \to |1\rangle \otimes g(f\, t)\}\}$$

Example 3 (Bounded quantum walk)**.** `Hyrql` *allows one to write recursive programs that are classically and quantumly controlled. The program* `bqwalk` *produces all the possible quantum walk paths of size at most n, starting from a quantum state q:*

$$\texttt{repeat} \triangleq \texttt{letrec}\, g\, n = \texttt{match}\, n\, \{0 \to [\,], S(m) \to |0\rangle :: g\, m\}$$

$$\texttt{bqwalk} \triangleq \texttt{letrec}\, f\, q = \lambda n.\texttt{qcase}\, q \left\{ \begin{array}{l} |0\rangle \to |0\rangle :: (\texttt{repeat}\ n) \\ |1\rangle \to |1\rangle :: \texttt{match}\, n \left\{ \begin{array}{c} 0 \to [\,] \\ S(m) \to (f\ (\texttt{Had}\ |1\rangle)\ m) \end{array} \right\} \end{array} \right\}$$

The programs from Examples 2 and 3 are strictly more than quantum circuits, as one execution of the program does not provide a full description of the corresponding behaviour. Thus, `Hyrql` is more than a circuit description language [28]. We also provide the hybrid quantum protocol BB84 [10] and the Quantum Fourier Transform in Appendix A.

2.2 Operational Semantics

In this section, we define the call-by-value operational semantics of `Hyrql`. As we are interested in resource analysis, we want to avoid reducing non-meaningful terms, e.g., a reduction of t in $s + 0 \cdot t \equiv s$ is meaningless. Towards that end,

we introduce *canonical forms* (Definition 4), which take care of removing ill-behaved terms before applying a reduction. It relies on *pure terms*, which are defined by the following grammar:

$$\text{(Pure terms)} \quad p ::= x \mid |0\rangle \mid |1\rangle \mid \texttt{qcase}\, p \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\} \mid c(p_1, \ldots, p_n) \\ \mid \texttt{match}_{1\le i\le n}\, p\, \{c_i(\overrightarrow{x_i}) \to t_i\} \mid \lambda x.t \mid \texttt{letrec}\, f\, x = t \mid tp \mid \texttt{shape}(t)$$

A *pure value*, denoted v_p, is a pure term that is a value.

Definition 4 (Canonical form)**.** *A canonical form of a term t is any term t' such that $t \equiv t'$ and $t' = \sum_{i=1}^{n} \alpha_i \cdot p_i$, where p_i are pure terms, $\alpha_i \neq 0$ and $p_i \equiv p_j$ implies $i = j$. The set of canonical forms is denoted by* CAN*.*

The canonical form always exists and is unique for well-typed terms (Lemma 12). Note that the naming *pure terms* comes from [24] and is not related to *pure states* in quantum computing, as pure terms could semantically yield a mixed quantum state.

We also define *evaluation contexts* by the following grammar:

$$\text{(Evaluation contexts)} \quad E ::= \diamond \mid \texttt{qcase}\, E \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\} \mid c(\overrightarrow{p}, E, \overrightarrow{v_p}) \\ \mid \texttt{match}_{1\le i\le n}\, E\, \{c_i(\overrightarrow{x_i}) \to t_i\} \mid tE \mid Ev_p$$

Again, let $E[t]$ be the term obtained by filling the hole $\diamond$ with t in E.

The operational semantics of Hyrql is described in Table 4 as a reduction relation $\rightsquigarrow \subseteq$ Terms $\times$ Terms, where $\{t/x\}$ denotes the standard substitution of a variable x by a term t. The reduction implements a *call-by-value* strategy.

In the rule (Can), $t \rightsquigarrow^{?} t'$ holds if either $t \rightsquigarrow t'$ or, $\neg(\exists t'',\ t \rightsquigarrow t'')$ and $t' = t$. Intuitively, it means that we apply one reduction for each element of a superposition, when possible. In the rule (Shape$_c$), $\texttt{shape}(\overrightarrow{v})$ is syntactic sugar for $\texttt{shape}(v_1), \ldots, \texttt{shape}(v_n)$, and the shape of a signature $\texttt{shape}(s)$ is defined formally in Definition 8. In all rules of Table 4, except for the (Can) and (Equiv) rules, the left-hand-side term is a pure term (i.e., not a superposition). This implies that any summation must be expressed as a canonical form, which avoids reducing a subterm with a null amplitude, or more generally to reduce two identical terms that would sum up to 0. The goal is to avoid reductions that have no physical meaning, as this would invalidate all resource analysis results.

We define $\rightsquigarrow^*$ as the reflexive and transitive closure of $\rightsquigarrow$. For $k \in \mathbb{N}$, we also write $t \rightsquigarrow^{\le k} t'$, when t reduces to t' in at most k steps. A term t *terminates*, if any chain of reduction starting from t reaches a value, meaning that $t \rightsquigarrow^{\le k} v$ holds for some k.

Example 5. *Let us consider the* Had *program from Example 1, we can check that it gives the desired result when applying it to $|0\rangle$:*

$$\begin{aligned} \texttt{Had}\,|0\rangle &\rightsquigarrow \texttt{qcase}\ |0\rangle \left\{|0\rangle \to |+\rangle\ , |1\rangle \to |-\rangle\right\} && \textit{via (Lbd)} \\ &\rightsquigarrow |+\rangle && \textit{via (Qcase}_0\textit{)} \end{aligned}$$

Since the Hadamard gate is its own inverse we can recover $|0\rangle$. Given the following term $h_v = \texttt{qcase}\, v \left\{|0\rangle \to |+\rangle\ , |1\rangle \to |-\rangle\right\}$, we have the following:

$$\texttt{Had}\,|+\rangle \equiv \frac{1}{\sqrt{2}}\cdot\texttt{Had}\,|0\rangle + \frac{1}{\sqrt{2}}\cdot\texttt{Had}\,|1\rangle \rightsquigarrow \frac{1}{\sqrt{2}}\cdot h_0 + \frac{1}{\sqrt{2}}\cdot h_1 \rightsquigarrow \frac{1}{\sqrt{2}}\cdot|+\rangle + \frac{1}{\sqrt{2}}\cdot|-\rangle \equiv |0\rangle$$

Note that the reductions are performed in parallel on each term of the superpositions using rule (Can) of Table 4.

Example 6. *Given a qubit list* $l = |0\rangle :: |1\rangle :: |+\rangle :: [\,]$*, one can verify that* $\texttt{shape}(l) \rightsquigarrow^* () :: () :: () :: [\,]$ *(note that* $::$ *and* $[\,]$ *use distinct signatures in* l *and the reduct of* $\texttt{shape}(l)$*). The result is a list of same length as* l*, with no quantum data anymore, hence it can be treated non-linearly. This operation is independent of the value of each qubit in the list.*

It is worth mentioning that while we can write qubit streams in `Hyrql`, the call-by-value strategy will require it to be evaluated before doing any operation on such data.

2.3 Type System

Types and Contexts. Types in `Hyrql` are provided by the following grammar:

$$\text{(Types)} \qquad T ::= \text{Qbit} \mid B \mid T \multimap T \mid T \Rightarrow T$$

Qbit is the type for qubits and B is a *constructor type* from a fixed set $\mathbb{B}$. A *basic type*, noted $B^{|\rangle}$, is either a qubit type, or a constructor type B, i.e., $B^{|\rangle} \in \{\text{Qbit}\} \cup \mathbb{B}$. Each constructor type is defined by its set of constructors:

$$\text{Cons}(B) \triangleq \{c^s \mid s = B_1^{|\rangle}, \ldots, B_n^{|\rangle} \to B\}$$

A constructor symbol c comes with a signature s of arity n, defining the inputs and output types of c. We may write c^B when s is of arity 0. By construction, $\text{Cons}(B) \cap \text{Cons}(B') = \emptyset$ when $B \neq B'$. In what follows, we will consider the following constructor types: a unit type $\mathbb{1}$ with a unique constructor $()^{\mathbb{1}}$; tensor types $B_1^{|\rangle} \otimes B_2^{|\rangle}$ for given types $B_1^{|\rangle}, B_2^{|\rangle}$ with a unique constructor $\otimes^{B_1^{|\rangle}, B_2^{|\rangle} \to B_1^{|\rangle} \otimes B_2^{|\rangle}}$; natural numbers $\texttt{nat}$ with constructors $0^{\texttt{nat}}$ and $S^{\texttt{nat} \to \texttt{nat}}$; lists $\texttt{list}(B^{|\rangle})$ for a given type $B^{|\rangle}$, with constructors $[\,]^{\texttt{list}(B^{|\rangle})}$ and $::^{B^{|\rangle}, \texttt{list}(B^{|\rangle}) \to \texttt{list}(B^{|\rangle})}$. We consider that $\mathbb{B}$ always contains the unit type $\mathbb{1}$.

The language also features two distinct *higher-order* types for linear and non-linear functions, denoted respectively by $\multimap$ and $\Rightarrow$.

In the typing discipline, it will be useful to distinguish between types based on whether or not they contain quantum data. The latter must follow the laws of quantum mechanics (no-cloning), whereas classical types are more permissive. Towards that end, we define inductively the set of *quantum constructor types* $\mathbb{B}_Q$ by

$$\mathbb{B}_Q \triangleq \{B \in \mathbb{B} \mid \exists\, c^{B_1^{|\rangle}, \ldots, B_n^{|\rangle} \to B}, \exists\, 1 \leq i \leq n, B_i^{|\rangle} \in \mathbb{B}_Q \cup \{\text{Qbit}\}\}.$$

A quantum constructor type $B_Q \in \mathbb{B}_Q$ has (at least) a constructor symbol with a type Qbit or (inductively) a quantum constructor type in its signature. The set $\mathbb{B}_C$ of *classical constructor types* B_C is defined by $\mathbb{B}_C \triangleq \mathbb{B} \setminus \mathbb{B}_Q$. For example, $\mathbb{1}$, $\texttt{nat} \in \mathbb{B}_C$ whereas $\texttt{list}(\text{Qbit}) \in \mathbb{B}_Q$. This allows us to split the types between *quantum types* Q and *classical types* C:

$$Q ::= \text{Qbit} \mid B_Q \qquad C ::= B_C \mid T \multimap T \mid T \Rightarrow T$$

It is now assumed that the inputs of each constructor are ordered, containing first variables of classical type, then variables of quantum type, enabling a hybrid behaviour.

$$\frac{}{\Gamma; x : T \vdash x : T}\ (\text{ax}) \qquad \frac{}{\Gamma, x : C; \varnothing \vdash x : C}\ (\text{ax}_c) \qquad \frac{}{\Gamma; \varnothing \vdash |0\rangle : \text{Qbit}}\ (\text{ax}_0)$$

$$\frac{}{\Gamma; \varnothing \vdash |1\rangle : \text{Qbit}}\ (\text{ax}_1) \qquad \frac{\Gamma; \Delta \vdash t : \text{Qbit} \qquad \Gamma; \Delta' \vdash t_0 : Q \qquad \Gamma; \Delta' \vdash t_1 : Q \qquad t_0 \perp t_1}{\Gamma; \Delta, \Delta' \vdash \texttt{qcase}\, t \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\} : Q}\ (\text{qcase})$$

$$\frac{s = B_1^{|\rangle}, \ldots, B_n^{|\rangle} \to B \qquad \Gamma; \Delta_i \vdash t_i : B_i^{|\rangle}}{\Gamma; \Delta_1, \ldots, \Delta_n \vdash c^s(t_1, \ldots, t_n) : B}\ (\text{cons})$$

$$\frac{\text{Cons}(B) = \{\, c_i^{s_i} \,\}_{i=1}^n \qquad s_i = \overrightarrow{C_i}, \overrightarrow{Q_i} \to B \qquad \Gamma_1; \Delta_1 \vdash t : B \qquad \Gamma_2, \overrightarrow{y_i} : \overrightarrow{C_i}; \Delta_2, \overrightarrow{z_i} : \overrightarrow{Q_i} \vdash t_i : B^{|\rangle}}{\Gamma_1, \Gamma_2; \Delta_1, \Delta_2 \vdash \texttt{match}_{1 \le i \le n}\, t \,\{ c_i^{s_i}(\overrightarrow{y_i}, \overrightarrow{z_i}) \to t_i \} : B^{|\rangle}}\ (\text{match})$$

$$\frac{\Gamma; \Delta, x : T \vdash t : T'}{\Gamma; \Delta \vdash \lambda x.t : T \multimap T'}\ (\text{abs}) \qquad \frac{\Gamma, x : C; \Delta \vdash t : T}{\Gamma; \Delta \vdash \lambda x.t : C \Rightarrow T}\ (\text{abs}_c) \qquad \frac{\Gamma, f : T; \varnothing \vdash \lambda x.t : T}{\Gamma; \varnothing \vdash \texttt{letrec}\, f\, x = t : T}\ (\text{rec})$$

$$\frac{\Gamma; \Delta \vdash t_1 : T \multimap T' \qquad \Gamma; \Delta' \vdash t_2 : T}{\Gamma; \Delta, \Delta' \vdash t_1 t_2 : T'}\ (\text{app}) \qquad \frac{\Gamma; \Delta \vdash t_1 : C \Rightarrow T \qquad \Gamma; \varnothing \vdash t_2 : C}{\Gamma; \Delta \vdash t_1 t_2 : T}\ (\text{app}_c)$$

$$\frac{\Gamma; \Delta \vdash t_i : Q \qquad \sum_{i=1}^n |\alpha_i|^2 = 1 \qquad \forall i \neq j,\ t_i \perp t_j}{\Gamma; \Delta \vdash \sum_{i=1}^n \alpha_i \cdot t_i : Q}\ (\text{sup}) \qquad \frac{\Gamma; \Delta \vdash t : B^{|\rangle}}{\Gamma, [\Delta]; \varnothing \vdash \texttt{shape}(t) : \texttt{shape}(B^{|\rangle})}\ (\text{shape})$$

$$\frac{\Gamma, [x : B^{|\rangle}]; \Delta, y : B^{|\rangle} \vdash t : T}{\Gamma; \Delta, y : B^{|\rangle} \vdash t\{y/x\} : T}\ (\text{contr}) \qquad \frac{\Gamma; \Delta \vdash t : T \qquad t \equiv t'}{\Gamma; \Delta \vdash t' : T}\ (\text{equiv})$$

Table 5: Typing rules of the language

Typing contexts are defined as follows:

$$\Gamma, \Delta \quad ::= \quad \varnothing \mid \{\, x : T \,\} \mid \{\, [x : T] \,\} \mid \Gamma \cup \Delta,$$

where $[x : T]$ is called a *boxed variable*, indicating that x is a linear variable captured by a `shape` construct. We define the *domain* of a context Δ as the set of its variables, i.e., $\text{dom}(\Delta) \triangleq \{\, x \mid \exists T,\, x : T \in \Delta \vee [x : T] \in \Delta \,\}$. Whenever we write Γ, Δ or $\Gamma; \Delta$, it is assumed that Γ and Δ are compatible, that is, $\text{dom}(\Gamma) \cap \text{dom}(\Delta) = \emptyset$. We use the shorthand notation $\overrightarrow{x} : \overrightarrow{T}$ for $x_1 : T_1, \ldots, x_n : T_n$; and $[\Delta]$ for $[x_1 : T_1], \ldots, [x_n : T_n]$.

Typing rules. A *typing judgment* is written $\Gamma; \Delta \vdash t : T$, describing that the term t is *well-typed*, with type T, under *non-linear context* Γ and *linear context* Δ. The typing rules of the language are defined in Table 5. A well-typed term $\Gamma; \Delta \vdash t : T$ is *closed* if $\Gamma = \Delta = \emptyset$. A well-typed closed term t is said to *terminate over any input*, if for any well-typed closed value $v_1, \ldots, v_n$ terminating over any input such that $tv_1 \ldots v_n$ is a well-typed closed term of type $B^{|\rangle}$, $tv_1 \ldots v_n$ terminates.

Typing rules rely on an external *orthogonality* predicate on terms. This uses the notion of Δ-*context substitutions*, which, for a given Δ, denote the substitutions σ such that for any $x : T$ with $x : T \in \Delta$ or $[x : T] \in \Delta$, $\sigma(x) = v$,

where v is a well-typed closed value of type T that terminates over any given input. It also requires the definition of an an *inner product* on values with a canonical form, defined below, where $\delta_{x,y}$ is the Kronecker symbol.

$$\langle v, w\rangle \triangleq \sum_{i=1}^{n}\sum_{j=1}^{m} \alpha_i \beta_j^* \delta_{v_i,w_j}, \qquad \text{where } v \equiv \sum_{i=1}^{n} \alpha_i \cdot v_i \in \texttt{CAN} \text{ and } w \equiv \sum_{j=1}^{m} \beta_j \cdot w_j \in \texttt{CAN}$$

Definition 7 (Orthogonality)**.** *Let* $\Gamma;\Delta \vdash t : Q$ *and* $\Gamma;\Delta \vdash t' : Q$ *be two well-typed terms.* t *and* t' *are* orthogonal*, written* $t \perp t'$*, if for any* $\Gamma \cup \Delta$*-context substitution* σ*:*

- $\sigma(\texttt{shape}(t)) \rightsquigarrow^* v$*,* $\sigma(\texttt{shape}(t')) \rightsquigarrow^* v'$ *and* $v = v'$*;*
- $\sigma(t) \rightsquigarrow^* v \in \texttt{CAN}$*,* $\sigma(t') \rightsquigarrow^* v' \in \texttt{CAN}$*, and* $\langle v, v'\rangle = 0$*.*

These conditions imply that the reduced values share the same classical structure, and that their quantum data is orthogonal. For example, orthogonal lists must be of the same size. This is used in the typing rules (qcase) and (sup) of Table 5 to ensure the feasibility of typed terms, similarly to existing languages [24, 37, 21]. Note that the rules requiring this predicate apply it only to well-typed terms, thus the type system is not cyclic. While an alternative condition could have been given with dependent types, this gives a uniform process that is defined for any type, including types that could be defined later by a programmer, and a simpler type system. As we will prove that values have a unique canonical form (Lemma 12), the inner-product condition is sound.

We also define the *shape of a basic type* below.

Definition 8. *The shape of a basic type is defined inductively as* $\texttt{shape}(\text{Qbit}) \triangleq \mathbb{1}$*, and* $\texttt{shape}(B)$ *is the type defined by the constructor set*

$$\text{Cons}(\texttt{shape}(B)) \triangleq \{\, c^{\texttt{shape}(s)} \mid c^s \in \text{Cons}(B) \,\}$$

where the shape of a signature $s = B_1^{|\rangle}, \ldots, B_n^{|\rangle} \to B$ *is defined as*

$$\texttt{shape}(s) \triangleq \texttt{shape}(B_1^{|\rangle}), \ldots, \texttt{shape}(B_n^{|\rangle}) \to \texttt{shape}(B).$$

In particular, the shape of a list of qubits is a list of units, i.e., $\texttt{shape}(\texttt{list}(\text{Qbit})) = \texttt{list}(\mathbb{1})$, which is coherent with the reduction rules in Table 4. By construction, it always holds that $\texttt{shape}(B^{|\rangle}) \in \mathbb{B}_C$. This is used in the typing rule (shape) of Table 5, which extracts the classical structure of a quantum term, boxes any variable in Δ, and puts it in the non-linear context. The judgment obtained has an empty linear context and thus can be used classically (i.e., duplicated or discarded). A boxed variable x can be removed using rule (contr) of Table 5, by associating with a linear variable of same type y, allowing to read both the quantum data and the classic structural information of y (see Example 9).

Typing derivations. We write $\pi \triangleright \Gamma;\Delta \vdash t : T$ for the tree of root $\Gamma;\Delta \vdash t : T$ derived using the rules of Table 5. We also denote by $\pi \rhd (\text{r})[t_1, \ldots, t_n]$ the subtree of π rooted by (r) and whose premises are $t_1, \ldots, t_n$.

Example 9. *The length of a list can be computed by the following term:*

$$\texttt{len} \triangleq \texttt{letrec}\, f\, x = \texttt{match}\, x\, \{[\,] \to 0, h :: t \to S(f\, t)\}$$

Notice that we discard h, thus the list is not used linearly. Therefore, we can only type this as a non-linear function, i.e., $\varnothing;\varnothing \vdash \texttt{len} : \texttt{list}(B) \Rightarrow \texttt{nat}$ for a given $B \in \mathbb{B}_C$. However, we can still compute the length of a qubit list with $\lambda x.(x \otimes \texttt{len}(\texttt{shape}(x)))$, whose typing derivation π is written below. For conciseness, we hide the signature of $\otimes$.

$$\dfrac{\dfrac{}{[y : \texttt{list}(\text{Qbit})]; x : \texttt{list}(\text{Qbit}) \vdash x : \texttt{list}(\text{Qbit})}\,(ax) \qquad \dfrac{\dfrac{\dfrac{}{\varnothing; y : \texttt{list}(\text{Qbit}) \vdash y : \texttt{list}(\text{Qbit})}\,(ax)}{[y : \texttt{list}(\text{Qbit})]; \varnothing \vdash \texttt{shape}(y) : \texttt{list}(\mathbb{1})}\,(shape)}{[y : \texttt{list}(\text{Qbit})]; \varnothing \vdash \texttt{len}(\texttt{shape}(y)) : \texttt{nat}}\,(app_c)}{\dfrac{[y : \texttt{list}(\text{Qbit})]; x : \texttt{list}(\text{Qbit}) \vdash x \otimes \texttt{len}(\texttt{shape}(y)) : \texttt{list}(\text{Qbit}) \otimes \texttt{nat}}{\dfrac{\varnothing; x : \texttt{list}(\text{Qbit}) \vdash x \otimes \texttt{len}(\texttt{shape}(x)) : \texttt{list}(\text{Qbit}) \otimes \texttt{nat}}{\varnothing;\varnothing \vdash \lambda x.(x \otimes \texttt{len}(\texttt{shape}(x))) : \texttt{list}(\text{Qbit}) \multimap \texttt{list}(\text{Qbit}) \otimes \texttt{nat}}\,(app)}\,(contr)}\,(cons)$$

As $\texttt{shape}(y)$ is of classical constructor type, it can be used as an input of $\texttt{len}$, allowing us to compute the length of y, given by the subtree $\pi \rhd (\text{app}_c)[\texttt{shape}(y)]$. The rule (contr) removes the marker introduced by (shape), by substituting y with a variable of same type, x, thus yielding a closed term. Note that while x appears twice in the final term, the quantum information of x has a linear behaviour, and only the classical structure is used non-linearly.

3 Main Results

This section is devoted to checking that `Hyrql` satisfies usual properties of typed languages. Moreover, it studies the complexity of checking the orthogonality predicate. We also prove that we can compile an expressive subset of terms of `Hyrql` into quantum circuits, with a guarantee on the size of the circuit. This gives us a characterization of the class of functions computable in quantum polynomial time, known as `FBQP` [11].

3.1 Standard Properties

By design of the type system, the non-linear context Γ can be weakened and typing still holds. This result does not hold for Δ, as intended, due to its linear behavior.

Proposition 10 (Weakening)**.** *Let $\Gamma;\Delta \vdash t : T$ be a well-typed term. Then $\Gamma,\Gamma';\Delta \vdash t : T$ holds for any context Γ' compatible with Γ,Δ.*

The reduction relation $\rightsquigarrow$ is confluent up to equivalence for well-typed terms.

Theorem 11 (Confluence)**.** *Given a well-typed term t, if there exist t_1 and t_2 such that $t \rightsquigarrow^* t_1$ and $t \rightsquigarrow^* t_2$, then there exist t_3 and t_4 such that $t_1 \rightsquigarrow^* t_3$, $t_2 \rightsquigarrow^* t_4$ and $t_3 \equiv t_4$.*

Due to rule (Equiv) of Table 4, confluence is up to equivalence $\equiv$. Moreover, confluence only holds on well-typed terms. A consequence of Theorem 11 is that any terminating term has a unique normal form, up to equivalence. This will be useful for progress (Theorem 13).

Lemma 12 (Canonical form for typed terms)**.** *Let $\Gamma;\Delta \vdash t : T$ be a well-typed term. Then t has a canonical form $\sum_{i=1}^{n} \alpha_i \cdot p_i$, and this canonical form is unique, up to equivalence on each p_i and term reordering. Furthermore, $\forall\ 1 \leq i \leq n,\ \Gamma;\Delta \vdash p_i : T$.*

As there are no distinct and equivalent pure values, Lemma 12 implies that the canonical forms of Definition 7 exist and are unique.

Typing implies that the values are the normal forms of the language, up to equivalence.

Theorem 13 (Progress)**.** *Let $\varnothing;\varnothing \vdash t : T$ be a closed term, either t is equivalent to a value, or t reduces.*

Contrary to standard progress lemmas, we require t to be a value up to equivalence. For instance, the term $|0\rangle + 0 \cdot (\lambda y.y)\,|1\rangle$ does not reduce and is not a value.

Finally, typing is also preserved by reduction, provided we consider pure or terminating terms.

Theorem 14 (Subject reduction)**.** *Let $\Gamma;\Delta \vdash t : T$ be a well-typed term, and $t \rightsquigarrow t'$. If t terminates or t is pure, then $\Gamma;\Delta \vdash t' : T$.*

Remark 15. *Theorem 14 does not hold on non-terminating and non-pure terms, as we cannot type non-terminating superpositions. This is the case with the following example, with the Bell state $|\Phi^+\rangle = \frac{1}{\sqrt{2}} \cdot |0\rangle \otimes |0\rangle + \frac{1}{\sqrt{2}} \cdot |1\rangle \otimes |1\rangle$:*

$$\left(\lambda y.\mathtt{qcase}\left((\mathtt{letrec}\, f\, x = f x)\,|0\rangle\right)\left\{|0\rangle \to |0\rangle \otimes y\,, |1\rangle \to |1\rangle \otimes y\right\}\right)|\Phi^+\rangle$$

While such a term is well-typed, it reduces to a superposition of non-terminating pure terms, which is impossible to type through the rule (sup) from Table 5, as the orthogonality predicate requires termination. This is a counterpart of having a general language with few restrictions; however, as resource analysis will be done on terminating terms, such ill-behaved cases will not be considered.

3.2 Decidability of Orthogonality

The orthogonality predicate of Definition 7 is extensionally complete. As a consequence, it is undecidable. Indeed, it is as hard as the Universal Halting Problem [25], that is Π_2^0-complete in the arithmetical hierarchy [41]. Orthogonality uses the inner product, which requires computing additions, multiplications, and nullity checks on complex numbers. Recall that algebraic numbers are complex numbers in $\mathbb{C}$ that are roots of a polynomial in $\mathbb{Q}[X]$, and write their sets as $\bar{\mathbb{C}}$. In the field of algebraic numbers, equality is decidable, and product and sum are computable in polynomial time [31]. We thus restrict ourselves to terms that only contain amplitudes $\alpha \in \bar{\mathbb{C}}$.

Theorem 16 (Undecidability of orthogonality)**.** *Deciding orthogonality between two well-typed terms is Π_2^0-complete.*

This is not surprising as `Hyrql` allows for general recursion. Decidability can be recovered under some slight restrictions. Towards that end, we introduce for basic types a *type depth* d as a partial function $d : \{\text{Qbit}\} \cup \mathbb{B} \to \mathbb{N}$, defined as follows:

$$d(\text{Qbit}) \triangleq 1 \qquad d(B) \triangleq \max_{\substack{s=B_1^{|\rangle},\dots,B_n^{|\rangle}\to B \\ c^s \in \text{Cons}(B)}} (\sum_{i=1}^{n} d(B_i^{|\rangle})) + 1$$

Note that $d(B^{|\rangle})$ is undefined on inductive types. We call *finite types* any basic type with a defined depth.

Proposition 17. *Given a finite type $B^{|\rangle}$, and two terminating terms $\varnothing;\varnothing \vdash s : B^{|\rangle}$, $\varnothing;\varnothing \vdash t : B^{|\rangle}$, it is decidable whether they are orthogonal.*

In previous works of `Spm` [15, 14], the syntax of the language was restrictive enough to ensure the decidability of orthogonality. As `Spm` is a strict sublanguage of `Hyrql`, orthogonality can also be decided on this fragment.

3.3 Complexity Properties

We finally discuss how quantum circuits and their size are related to terms of `Hyrql` and their runtime-complexity. Towards that end, we introduce a sublanguage of `Hyrql` (`Hyrql`(T), Definition 24) based on three restrictions: one on the runtime-complexity of the program, to ensure terminating terms (`Time`(T), Definition 18); one on types, to obtain terms that can be represented as circuits (`CircuitTerms`, Definition 19); and one on recursive calls, to have a faithful relation between the size of the circuit and the runtime-complexity (`Faithful`, Definition 22). This allows, for terms terminating in polynomial time, to characterize precisely the class `FBQP`. We first define the time termination of a term.

Definition 18. *Given a term $\varnothing;\varnothing \vdash t : Q \multimap Q'$ and a function $T : \mathbb{N} \to \mathbb{N}$, we say that t* terminates in time T*, if for any value $\varnothing;\varnothing \vdash v : Q$, tv terminates in at most $T(|v|)$ steps. Let* `Time`(T) *be the set of terms terminating in time T.*

Note that this definition should not be confused with termination defined in Section 2.2.

To obtain terms that can be compiled to a circuit form, we introduce a typed restriction `CircuitTerms`, which relies on the following type grammar:

$$B_r ::= \text{Qbit} \mid \texttt{nat} \mid \texttt{list}(B_r) \mid B_r \otimes B_r \mid B_r \multimap B_r \mid B_r \Rightarrow B_r$$

Definition 19 (Circuit terms)**.** *We define* `CircuitTerms` *as the set of well-typed terms $\pi \triangleright \varnothing;\varnothing \vdash s : Q \multimap Q'$ satisfying the following conditions:*

- *Any type in π is generated by the grammar of B_r;*
- *Any rule $\pi \rhd (\text{sup})[t_1,\dots,t_n]$ satisfies $t_i \in \text{Value}$;*
- *Any rule $\pi \rhd (\text{qcase})[t,t_0,t_1]$ satisfies either $t_0,t_1 \in \text{Value}$; or $t_0 = c(|0\rangle, s_0)$ and $t_1 = c(|1\rangle, s_1)$, for $c \in \{\otimes, ::\}$.*

While the first condition yields terms that can be represented by a circuit, the two others allow for an efficient compilation and guarantees that any superposition can be compiled effectively to circuits. Note that while these terms

only take inputs of type Q, to represent them clearly in a quantum circuit, they can nonetheless contain multiple inputs and higher-order constructs in their body. This restriction alone is sufficient to compile t to a family of circuits in Theorem 21, provided t terminates for a given time T.

Example 20. *The following term, which is an adaptation of the program from Example 3, belongs to* `CircuitTerms`.

$$\pi \triangleright \varnothing; \varnothing \vdash \lambda x.\mathtt{match}\, x \left\{ q \otimes n \to \mathtt{bqwalk}\, q\, n \right\} : \mathrm{Qbit} \otimes \mathtt{nat} \multimap \mathtt{list}(\mathrm{Qbit})$$

Any type in π is derived from B_r. Term superposition happens only in `Had` *and is between values. Furthermore, there are two qcase constructs, either between values, or between two terms $|0\rangle :: s_0$ and $|1\rangle :: s_1$, thus it satisfies all properties of* `CircuitTerms`. *Note that as B_r allows for tensor products, any program taking multiple inputs can always be rewritten using the same technique so that it belongs to* `CircuitTerms`.

In the literature, family of circuits are parameterized by the size of their input. However, in `Hyrql`, knowing the size might not be sufficient, e.g., on inputs of type $\mathtt{list}(\mathrm{Qbit}) \otimes \mathtt{list}(\mathrm{Qbit})$. To remedy this, we will parameterize the family by the *shape* of the input, which will give us the necessary information. Formally, we define

$$\mathtt{S}(Q) \triangleq \{\, w \mid \varnothing; \varnothing \vdash w : \mathtt{shape}(Q) \,\}$$
$$\mathtt{S}(w) \triangleq \{\, v \mid \varnothing; \varnothing \vdash v : Q \wedge \mathtt{shape}(v) \rightsquigarrow^* w \,\}, \forall w \in \mathtt{S}(Q),$$

where $\mathtt{S}(Q)$ is the *set of shape of terms of type* Q, and $\mathtt{S}(w)$ the *set of values of shape* w.

Assuming an encoding of $t \in \mathtt{CircuitTerms}$ to a quantum state $|t\rangle$, which can be done naturally, a circuit C is said to *approximate* v on input v' with probability p, if C evaluates to $|\phi\rangle$, on input $|v'\rangle$ (up to ancillary qubits), and $|\langle v|\phi\rangle|^2 \geq p$.

Theorem 21 (Circuit compilation)**.** *Let $t \in \mathtt{CircuitTerms} \cap \mathtt{Time}(T)$ be a term of type $Q \multimap Q'$ for a given $T : \mathbb{N} \to \mathbb{N}$. Then, we can generate a family of circuits $\mathtt{C}(t) \triangleq (C_w)_{w \in \mathtt{S}(Q)}$ such that for any $w \in \mathtt{S}(Q)$ and any $v \in \mathtt{S}(w)$, C_w approximates v' with probability $\frac{2}{3}$ on input v, where $tv \rightsquigarrow^* v'$.*

We are now interested in the *size* of each circuit of the family, i.e., the total number of gates acting on the circuit; in particular, the goal is to exhibit a relation between the size and the runtime of the initial term t. To avoid a blowup due to recursive calls, we introduce a syntactic restriction `Faithful`. It relies on the notion of the *width* of a variable f in a term t, written $w(f,t)$ and defined inductively as follows:

$$w(f,x) \triangleq \delta_{x,f} \qquad w(f,|i\rangle) \triangleq 0 \qquad w(f,\lambda x.t) \triangleq w(f,t) \qquad w(f,\mathtt{letrec}\, g\, x = t) \triangleq w(f,t)$$
$$w(f,t_1 t_2) \triangleq w(f,t_1) + w(f,t_2) \qquad w(f,\textstyle\sum_{i=1}^n \alpha_i \cdot t_i) = \max_{1\leq i\leq n} w(f,t_i)$$
$$w(f,c(t_1,\ldots,t_n)) \triangleq \textstyle\sum_{i=1}^n w(f,t_i) \qquad w(f,\mathtt{shape}(t)) = w(f,t)$$
$$w(f,\mathtt{qcase}\, t \left\{ |0\rangle \to t_0\,, |1\rangle \to t_1 \right\}) \triangleq w(f,t) + \max(w(f,t_0), w(f,t_1))$$
$$w(f,\mathtt{match}_{1\leq i\leq n}\, t \left\{ c_i(\overrightarrow{x_i}) \to t_i \right\}) \triangleq w(f,t) + \max_{1\leq i\leq n}(w(f,t_i))$$

Definition 22. *A term t is said to* compile faithfully*, if for any term $t' = (\texttt{letrec}\, f\, x = s)$ such that $t' \trianglelefteq t$, the following conditions hold:*

- $w(f, s) \leq 1$;
- $\exists!\ t', \forall\ t_1, f t_1 \trianglelefteq s \implies t_1 = t'$;
- $\neg \exists\ t', t' f \trianglelefteq s$.

Let `Faithful` *be the set of terms that compile faithfully.*

The first two conditions are derived from [30], and ensure that, for each recursive program $\texttt{letrec}\, f\, x = s$, all recursive calls are done in different branches of a `qcase` or `match`, with the same input. Furthermore, the last condition ensures that this behaviour is preserved, even in presence of higher-order constructs.

Example 23. *The program of Example 20 belongs to* `Faithful`*, as both* `bqwalk` *and* `repeat` *only perform one recursive call. All the examples across the paper also satisfy this criterion.*

Definition 24. *Let $T : \mathbb{N} \to \mathbb{N}$. We define* $\texttt{Hyrql}(T)$ *as follows:*

$$\texttt{Hyrql}(T) \triangleq \texttt{CircuitTerms} \cap \texttt{Faithful} \cap \texttt{Time}(T)$$

Any term in this subset compiles to quantum circuits of size bounded polynomially by T.

Theorem 25 (Bounded circuit compilation)**.** *Let $T : \mathbb{N} \to \mathbb{N}$, and let $t \in \texttt{Hyrql}(T)$ be a term of type $Q \multimap Q'$. Then, we can generate a family of circuits $\texttt{C}(t) \triangleq (C_w)_{w \in \texttt{S}(Q)}$ and a polynomial P such that, for any $w \in \texttt{S}(Q)$:*

- $|C_w| \leq P(T(|w|))$;
- $\forall\ v \in \texttt{S}(w)$, *$C_w$ approximates v' with probability $\frac{2}{3}$ on input v, where $tv \rightsquigarrow^* v'$.*

We now want to use Theorem 25 to obtain a characterization of `FBQP` (Definition 26). We recall that a family of circuits $(C_n)_{n \in \mathbb{N}}$ is said to be *uniform polynomially-sized* with $\alpha : \mathbb{N} \to \mathbb{N}$ if there exists $P \in \mathbb{N}[X]$ such that $|C_n| \leq P(n)$ and C_n has exactly $\alpha(n)$ ancillary qubits, and there is a polynomial-time Turing machine, which takes n as input, and outputs a representation of C_n for any $n \in \mathbb{N}$.

Definition 26 ([11])**.** *A binary function $f : \{0,1\}^* \to \{0,1\}^*$ is said to be computed by a family of circuits $(C_n)_{n \in \mathbb{N}}$ if for any $x \in \{0,1\}^*$, $C_{|x|}$ approximates $f(x)$ with probability $\frac{2}{3}$ on input x.* `FBQP` *is defined as the set of binary functions computed by a uniform polynomially-sized family of circuits.*

To characterize `FBQP`, we define the following set of terms, which terminate in polynomial time, and have list of qubits as input:

$$\texttt{Hyrql}(\text{Poly})_{\texttt{QList}} \triangleq \cup_{P \in \mathbb{N}[X]} \{\, t \in \texttt{Hyrql}(P) \mid \varnothing; \varnothing \vdash t : \texttt{list}(\text{Qbit}) \multimap Q \,\}$$

These generated families of circuits will be indexed by natural numbers instead of shape, as one can remark that $\texttt{S}(\texttt{list}(\text{Qbit}))$ is in bijection with $\mathbb{N}$. Altogether, $\texttt{Hyrql}(\text{Poly})_{\texttt{QList}}$ contain terms that are compiled to circuits approximating functions of `FBQP`.

Theorem 27 (FBQP soundness)**.** *Let* $t \in \texttt{Hyrql}(\text{Poly})_{\texttt{QList}}$*, and let* $f : \{0,1\}^* \to \{0,1\}^*$*. If* f *is computed by* $\texttt{C}(t)$*, then* $f \in \texttt{FBQP}$*.*

For example, the Quantum Fourier Transform program from Example 31 belongs to FBQP, and can thus be compiled to circuits of polynomial size.

We now prove FBQP completeness, meaning that every function of FBQP can be represented by a function in $\texttt{Hyrql}(\text{Poly})_{\texttt{QList}}$. This is proven by writing any function of Yamakami's algebra [53], which is known to capture FBQP, into $\texttt{Hyrql}(\text{Poly})_{\texttt{QList}}$.

Theorem 28 (FBQP completeness)**.** *Let* $f : \{0,1\}^* \to \{0,1\}^* \in \texttt{FBQP}$*. Then, there exists* $t \in \texttt{Hyrql}(\text{Poly})_{\texttt{QList}}$ *such that* $\texttt{C}(t)$ *computes* f*.*

We have given a typed, syntactic and semantic restriction of Hyrql, that compiles into a bounded family of quantum circuits. Such restriction could be refined, either to accept more terms by lifting requirements in Definition 22, as done in [30]; or by adding requirements to characterize more precise complexity classes [27]. We are nonetheless able to encapsulate FBQP, which demonstrates the expressivity of the $\texttt{Hyrql}(\text{Poly})_{\texttt{QList}}$ fragment.

4 Towards Automatization

Theorem 25 establishes that, given a program that terminates with a specified complexity bound, one can extract a concrete quantum circuit exhibiting the same space complexity. This result is particularly significant, as it enables the characterization of quantum polynomial time, as shown in Theorem 27 and Theorem 28. However, the burden of proving termination of a Hyrql program currently rests entirely on the programmer. This naturally raises the following question: to what extent is the termination of Hyrql programs semi automatable?

We argue that such automatization is feasible by leveraging techniques from Term Rewrite Systems (TRS)[1], as they are equipped with a rich toolbox for analyzing both termination and computational complexity. Approaches to termination analysis include non-exhaustively recursive path orderings [22, 23], dependency pairs [4], and size-change termination [36]. Similarly, several methods address complexity analysis, e.g., interpretation methods [13, 38, 7], and polynomial path orders [39, 5].

Although a full formal treatment is left to future work, we present an illustrative example supporting our claim. Specifically, we illustrate how a Hyrql program can be efficiently translated into a (quantum) TRS, how the reduction behaviour of the original program is faithfully simulated by the resulting rewriting system while preserving complexity bounds, and how termination and complexity results established for the TRS can be transferred back to the corresponding Hyrql program.

For example, the standard NOT gate is represented by the quantum TRS whose set of rewrite rules is $\{\texttt{NOT}\ |0\rangle \to |1\rangle, \texttt{NOT}\ |1\rangle \to |0\rangle\}$. The application of the NOT function symbol to $|0\rangle$ rewrites to $|1\rangle$. Similarly, the Hadamard gate H can be defined using a superposition of terms $\{\texttt{H}\ |0\rangle \to \frac{1}{\sqrt{2}} \cdot |0\rangle + \frac{1}{\sqrt{2}} \cdot |1\rangle, \texttt{H}\ |1\rangle \to$

[1] More specifically, a higher-order version called Simply-Typed TRS [52], but we do not enter this level of details here.

$\frac{1}{\sqrt{2}} \cdot |0\rangle - \frac{1}{\sqrt{2}} \cdot |1\rangle\}$. The evaluation of a quantum term will follow a parallel strategy, similar to `Hyrql` operational semantics (Table 4), which reduces in parallel the distinct branches of a superposition, when possible (otherwise, the reduction will have an exponential blowup).

We claim that we can automatize the translation of any term of `Hyrql` to a corresponding TRS. This translation works as follows:

- Produce a set of rules, inductively on the syntax of the term;
- Yield a function symbol `f` for the initial term t;
- When encountering $\texttt{qcase}\, x \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\}$ (same for `match`), merge the two sets of rules generated for t_0 and t_1, while keeping track of which path was taken.

Coming back on Example 3, such an algorithm would generate the following rewrite rules:

$$\left\{ \begin{array}{cc} \texttt{Repeat}\, 0 \to [\,] & \texttt{BQWalk}\ |0\rangle\ m \to |0\rangle :: \texttt{Repeat}\, m \\ \texttt{Repeat}\, S(m) \to |0\rangle :: \texttt{Repeat}\, m & \texttt{BQWalk}\ |1\rangle\ 0 \to |1\rangle :: [\,] \\ & \texttt{BQWalk}\ |1\rangle\ S(m) \to |1\rangle :: \texttt{BQWalk}\, (|-\rangle)\, m \end{array} \right\}$$

One can remark that this set of rules captures completely the behaviour of `bqwalk`, and they both compute the same function; furthermore, one reduction step for a given rewrite rule will correspond to $1 \leq k \leq |\texttt{bqwalk}|$ reduction steps in `Hyrql`. Therefore, a certificate on this TRS yields directly a certificate on the original term.

Now, it is possible to apply the tools we mentioned to the obtained TRS. In particular, we can use recursive path orderings [22] to show termination, coupled with quasi-interpretations [13] to get that this TRS terminates in time $\mathcal{O}(n)$ for any input of size n. While such properties are not defined for quantum TRS, i.e., with superpositions and parallel reduction, they can be adapted to fit the quantum setting; a formal adaptation is left for future work. Altogether with Theorem 25, we can conclude that `bqwalk` can be compiled to a family of quantum circuits, whose size is polynomial in the size of the input.

Using the same ideas, we claim that we can describe a semantic-preserving translation algorithm from `Hyrql` to TRS, with no blowup on the termination complexity in either direction. Such work, along with a proper development of TRS in the quantum setting and an adaptation of existing techniques, would give a semi-automatized way to prove the feasibility/tractability of terms in `Hyrql`.

While this section was focused on termination and time complexity, some of the mentioned techniques also provide results to bound the space complexity of a TRS program [13]. Given a good definition of quantum space, this would make it possible to characterize quantum space complexities, and obtain bounds on the depth or width of the circuit.

A Additional Material

Definition 29 (Size of a term)**.** *Given a term t, its size, written $|t| \in \mathbb{N}$, is defined inductively as follows:*

$$|x| \triangleq 1 \qquad |\,|0\rangle\,| \triangleq 1 \qquad |\,|1\rangle\,| \triangleq 1 \qquad |\mathtt{qcase}\, t\, \{|0\rangle \to t_0\,, |1\rangle \to t_1\}\,| \triangleq 1 + |t| + |t_0| + |t_1|$$

$$|c(t_1, \ldots, t_n)| \triangleq 1 + \textstyle\sum_{i=1}^{n} |t_i| \qquad |\mathtt{match}_{1 \le i \le n}\, t\, \{c_i(\overrightarrow{x_i}) \to t_i\}| \triangleq 1 + |t| + \textstyle\sum_{i=1}^{n} |t_i|$$

$$|\lambda x.t| \triangleq 1 + |t| \qquad |\mathtt{letrec}\, f\, x = t| \triangleq 1 + |t| \qquad |t_1 t_2| \triangleq |t_1| + |t_2|$$

$$|\textstyle\sum_{i=1}^{n} \alpha_i \cdot t_i| \triangleq 1 + \textstyle\sum_{i=1}^{n} |t_i| \qquad |\mathtt{shape}(t)| \triangleq 1 + |t|$$

Example 30 (Hybrid function — *BB84* [10])**.** `Hyrql` *is able to use quantum and classical data simultaneously. For example, the well-known protocol* BB84 *[10], creating a quantum key from a classical key, can be implemented as follows:*

$$\begin{aligned}
\mathtt{not} &\triangleq \lambda q.\mathtt{qcase}\, q\, \{|0\rangle \to |1\rangle\,, |1\rangle \to |0\rangle\} \\
\mathtt{cc} &\triangleq \lambda b.\lambda f.\lambda q.\mathtt{match}\, b\, \{0 \to q, 1 \to f\, q\} \\
\mathtt{op} &\triangleq \lambda q.\lambda n.\mathtt{match}\, n\, \{(x, h) \to \mathtt{cc}\, h\, \mathtt{Had}(\mathtt{cc}\, x\, \mathtt{not}\, q)\} \\
\mathtt{keygen} &\triangleq \mathtt{letrec}\, f\, l = \mathtt{match}\, l\, \{[\,] \to [\,], h :: t \to (\mathtt{op}\, |0\rangle\, h) :: (f\, t)\}
\end{aligned}$$

The term `cc` *applies a gate f conditionally with classical control on a bit b. The term* `op` *reads a pair (x, h), and applies the NOT gate if $x = 1$, and then applies the Hadamard gate if $h = 1$. Finally,* `keygen` *generates the qubit list key.*

Example 31 (Quantum Fourier Transform)**.** *We can encode the well-known Quantum Fourier Transform (QFT) in our language, with however two points to notice:*

- *As our natural numbers are defined recursively, defining phase gates is not direct. We thus assume that we are given a term ϕ such that $\phi\, n\, |0\rangle = |0\rangle$ and $\phi\, n\, |1\rangle = e^{i\pi/2^{n-1}}\, |1\rangle$.*
- *We will use the syntactic sugar to split 2 qubits and rename them:*

$$\mathtt{let}\, x \otimes y = s\, \mathtt{in}\, t \triangleq \mathtt{match}\, s\, \{x \otimes y \to t\}$$

$$\begin{aligned}
\mathtt{cphase} &\triangleq \lambda x.\lambda n.\mathtt{match}\, x\, \{c \otimes t \to \mathtt{qcase}\, c\, \{|0\rangle \to |1\rangle \otimes t\,, |1\rangle \to |0\rangle \otimes \phi\, n\, t\}\} \\
\mathtt{rot} &\triangleq \mathtt{letrec}\, f\, q = \lambda l.\lambda n.\mathtt{match}\, l \begin{Bmatrix} [\,] \to q \otimes [\,], h :: t \to \mathtt{let}\, h' \otimes q' = \mathtt{cphase}(h \otimes q)(n)\, \mathtt{in} \\ \mathtt{let}\, q'' \otimes t' = f(q')(t)(S(n))\, \mathtt{in}\, q'' \otimes (h' :: t') \end{Bmatrix} \\
\mathtt{rec} &\triangleq \mathtt{letrec}\, f\, l = \mathtt{match}\, l\, \{[\,] \to [\,], h :: t \to \mathtt{let}\, h' \otimes t' = \mathtt{rot}(\mathtt{had}\, h)(t)(S(S(0)))\, \mathtt{in}\, h' :: (f\, t')\} \\
\mathtt{append} &\triangleq \mathtt{letrec}\, f\, l = \lambda x.\mathtt{match}\, l\, \{[\,] \to x :: [\,], h :: t \to h :: f\, t\, x\} \\
\mathtt{swap} &\triangleq \mathtt{letrec}\, f\, l = \mathtt{match}\, l\, \{[\,] \to [\,], h :: t \to \mathtt{append}(f\, t)\, h\} \\
\mathtt{qft} &\triangleq \lambda l.\mathtt{swap}(\mathtt{rec}(l))
\end{aligned}$$

For the sake of simplicity in the following typing trees, we may denote $;\vdash t : T$ for $\varnothing; \varnothing \vdash t : T$.

Example 32. *As* $|0\rangle, |1\rangle$ *are pure values, then we get* $|0\rangle \perp |1\rangle$ *directly from the definition. This allows us to type* $|\pm\rangle$*:*

$$\dfrac{\dfrac{}{;\vdash |0\rangle : \mathrm{Qbit}}\ (ax_0) \qquad \dfrac{}{;\vdash |1\rangle : \mathrm{Qbit}}\ (ax_1) \qquad |\tfrac{1}{\sqrt{2}}|^2 + |\pm\tfrac{1}{\sqrt{2}}|^2 = 1 \qquad |0\rangle \perp |1\rangle}{;\vdash |\pm\rangle : \mathrm{Qbit}}\ (sup)$$

It is also easy to check that $|+\rangle \perp |-\rangle$*:*

$$\begin{aligned}\langle |+\rangle, |-\rangle \rangle &= \frac{1}{\sqrt{2}}\frac{1}{\sqrt{2}}\delta_{|0\rangle,|0\rangle} - \frac{1}{\sqrt{2}}\frac{1}{\sqrt{2}}\delta_{|0\rangle,|1\rangle} + \frac{1}{\sqrt{2}}\frac{1}{\sqrt{2}}\delta_{|1\rangle,|0\rangle} - \frac{1}{\sqrt{2}}\frac{1}{\sqrt{2}}\delta_{|1\rangle,|1\rangle} \\ &= \frac{1}{2} - \frac{1}{2} = 0\end{aligned}$$

we can also type the Hadamard gate as follows:

$$\dfrac{\dfrac{\dfrac{}{\varnothing; x : \mathrm{Qbit} \vdash x : \mathrm{Qbit}} \qquad ;\vdash |+\rangle : \mathrm{Qbit} \qquad ;\vdash |-\rangle : \mathrm{Qbit} \qquad |+\rangle \perp |-\rangle}{\varnothing; x : \mathrm{Qbit} \vdash \mathtt{qcase}\, x \left\{|0\rangle \to |+\rangle, |1\rangle \to |-\rangle\right\} : \mathrm{Qbit}}\ (qcase)}{;\vdash \mathtt{Had} : \mathrm{Qbit} \multimap \mathrm{Qbit}}\ (lbd)$$

Example 33. *The quantum switch from Example 2 can be typed by our language. For conciseness, we write* $v = |0\rangle \otimes f(g\,t)$*,* $w = |1\rangle \otimes g(f\,t)$*,* $\Gamma = f : \mathrm{Qbit} \multimap \mathrm{Qbit}, g : \mathrm{Qbit} \multimap \mathrm{Qbit}$*,* $s = \mathtt{match}\, q \left\{(c \otimes t) \to \mathtt{qcase}\, c \left\{|0\rangle \to v, |1\rangle \to w\right\}\right\}$*,* $\Delta = c : \mathrm{Qbit}, t : \mathrm{Qbit}$ *and* $Q_2 = \mathrm{Qbit} \otimes \mathrm{Qbit}$*. The orthogonality predicate* $v \perp w$ *is verified, as for any substitution of* f, g, t*,* $\sigma(v) \rightsquigarrow^* |0\rangle \otimes v'$ *and* $\sigma(w) \rightsquigarrow^* |1\rangle \otimes w'$*. Both terms have the same shape* $() \otimes ()$ *and their inner product is zero. Note that we substitute by terminating terms, thus* f, g *terminate and we can evaluate the inner product.*

$$\dfrac{\dfrac{\dfrac{\dfrac{\Gamma; c : \mathrm{Qbit} \vdash c : \mathrm{Qbit} \qquad \Gamma; t : \mathrm{Qbit} \vdash v : Q_2 \qquad \Gamma; t : \mathrm{Qbit} \vdash w : Q_2 \qquad v \perp w}{\Gamma; \Delta \vdash \mathtt{qcase}\, c \left\{|0\rangle \to v, |1\rangle \to w\right\} : Q_2} \qquad \Gamma; q : Q_2 \vdash q : Q_2}{\Gamma; q : Q_2 \vdash s : Q_2}}{\Gamma; \varnothing \vdash \lambda q.s : Q_2 \multimap Q_2}}{f : Q_2 \multimap Q_2; \varnothing \vdash \lambda g.\lambda q.s : (\mathrm{Qbit} \multimap \mathrm{Qbit}) \Rightarrow (Q_2 \multimap Q_2)}}{;\vdash \mathtt{QS} : (\mathrm{Qbit} \multimap \mathrm{Qbit}) \Rightarrow (\mathrm{Qbit} \multimap \mathrm{Qbit}) \Rightarrow (Q_2 \multimap Q_2)}$$

Example 34. *Coming back on Example 9, the following typing tree derives typing for* `len`*, for a given type* $A \in \mathbb{B}_C$*, with the following notations to shorten the tree:* $\Gamma_f = f : \mathtt{list}(A) \Rightarrow \mathtt{nat}$*,* $\Gamma_x = x : \mathtt{list}(A)$*, and* $\Gamma = \Gamma_f, \Gamma_x, h : A, t : \mathtt{list}(A)$*. For conciseness, we also do not type* f *and* t *on the right part of the tree, which can both be typed as* $\Gamma; \varnothing \vdash f : \mathtt{list}(A) \Rightarrow \mathtt{nat}$ *and* $\Gamma; \varnothing \vdash t : \mathtt{list}(A)$*, using* (ax_c) *rules. We also drop the signature on constructors once they are fully applied.*

$$\dfrac{\dfrac{}{\Gamma_f, \Gamma_x; \varnothing \vdash x : \texttt{list}(A)}\,(ax) \quad \dfrac{}{\Gamma_f, \Gamma_x; \varnothing \vdash 0^{\texttt{nat}} : \texttt{nat}}\,(cons) \quad \dfrac{\dfrac{}{\Gamma; \varnothing \vdash ft : \texttt{nat}}\,(app_c)}{\Gamma; \varnothing \vdash S^{\texttt{nat}\to\texttt{nat}}(ft) : \texttt{nat}}\,(cons)}{\Gamma_f, \Gamma_x; \varnothing \vdash \texttt{match}\, x \left\{[\,] \to 0 \,, h :: t \to S(ft)\right\} : \texttt{nat}}\,(match)$$

$$\dfrac{\Gamma_f, \Gamma_x; \varnothing \vdash \texttt{match}\, x \left\{[\,] \to 0 \,, h :: t \to S(ft)\right\} : \texttt{nat}}{\Gamma_f; \varnothing \vdash \lambda x.\texttt{match}\, x \left\{[\,] \to 0 \,, h :: t \to S(ft)\right\} : \texttt{list}(A) \Rightarrow \texttt{nat}}\,(abs_c)$$

$$\dfrac{\Gamma_f; \varnothing \vdash \lambda x.\texttt{match}\, x \left\{[\,] \to 0 \,, h :: t \to S(ft)\right\} : \texttt{list}(A) \Rightarrow \texttt{nat}}{;\vdash \texttt{letrec}\, f\, x = \texttt{match}\, x \left\{[\,] \to 0 \,, h :: t \to S(ft)\right\} : \texttt{list}(A) \Rightarrow \texttt{nat}}\,(rec)$$

In particular, as h is not present in the syntax of $S(f\,t)$, we could not type `len` *if h must be used linearly, thus if $A \in Q$.*

Example 35. *Assuming we have a bit type* `bit` *for both constructors* $0 :: \texttt{bit}$ *and* $1 :: \texttt{bit}$*, the typing of Example 30 can be derived, by combining most of the techniques exhibited from the previous examples, giving us* $;\vdash \texttt{keygen} : \texttt{list}(\texttt{bit}) \Rightarrow \texttt{list}(\text{Qbit})$*. Note that when we used programs in the syntax of other terms, such as* `Had`*, it is just notation to avoid rewriting the whole term.*

B Proofs for Section 3.1

B.1 Intermediate results

In this subsection, we state some intermediate results and definitions that we will use through the main proofs. Furthermore, we may use the notation $\{x \to v\}$ for a substitution $\{v/x\}$, and define the *support* of a substitution $\text{Supp}(\sigma)$ as the set of variables x_i such that there exists t_i with $\{t_i/x_i\} \in \sigma$. When we do not need precision on the linearity of a functional term, we may use $\rightarrowtail \triangleq \multimap \cup \Rightarrow$. We also denote the free variables of t by $\text{FV}(t)$.

Proposition 10 (Weakening)**.** *Let $\Gamma; \Delta \vdash t : T$ be a well-typed term. Then $\Gamma, \Gamma'; \Delta \vdash t : T$ holds for any context Γ' compatible with Γ, Δ.*

Proof. This can be proven by induction on t, as the axiom rules satisfy this weakening property. Furthermore, the only variable removed from the non-linear context are bound variables, thus Γ' stays untouched at each point. □

Lemma 36. *Let $T \in \mathcal{T}$. Then, there exists a number $n \in \mathbb{N}$, types T_i for $1 \leq i \leq n$, and a base type $B^{|\rangle}$ such that $T = T_1 \rightarrowtail \ldots \rightarrowtail T_n \rightarrowtail B^{|\rangle}$.*

Proof. Direct by induction on T: either it is a base type itself, thus we take $n = 0$; or $T = T_1 \rightarrowtail T'$, then we use the induction hypothesis on T' and right associativity to conclude. □

We also introduce an operator $\theta_p(t)$, called the *quantity*, which aims to measure the amplitude of a pure term p in a term t, e.g., a superposition.

Definition 37. *Let p be a pure term. We define the following map $\theta_p : \Lambda \to \mathbb{C}$ as follows:*

$$\theta_p(x) \triangleq \delta_{p,x} \qquad \theta_p(|0\rangle) \triangleq \delta_{p,|0\rangle} \qquad \theta_p(|1\rangle) \triangleq \delta_{p,|1\rangle}$$
$$\theta_p(\mathtt{qcase}\, t' \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\}) \triangleq \delta_{p,\mathtt{qcase}\, p' \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\}} \theta_{p'}(t')$$
$$\theta_p(c(t_1, \ldots, t_n)) \triangleq \delta_{p,c(p_1,\ldots,p_n)} \times_{i=1}^{n} \theta_{p_i}(t_i)$$
$$\theta_p(\mathtt{match}_{1\leq i\leq n}\, t' \, \{c_i(\overrightarrow{x_i}) \to t_i\}) \triangleq \delta_{p,\mathtt{match}_{1\leq i\leq n}\, p' \, \{c_i(\overrightarrow{x_i})\to t_i\}} \theta_{p'}(t')$$
$$\theta_p(\lambda x.t) \triangleq \delta_{p,\lambda x.t} \qquad \theta_p(\mathtt{letrec}\, f\, x = t) \triangleq \delta_{p,\mathtt{letrec}\, f\, x=t} \qquad \theta_p(t_1 t_2) \triangleq \delta_{p,p_1p_2} \nu_{p_1,t_1} \nu_{p_2,t_2}$$
$$\theta_p(\textstyle\sum_{i=1}^{n} \alpha_i \cdot t_i) \triangleq \sum_{i=1}^{n} \alpha_i \cdot \theta_p(t_i) \qquad \theta_p(\mathtt{shape}(t')) \triangleq \delta_{p,\mathtt{shape}(p')} \nu_{p',t'}$$

We have used the notation $\delta_{p,t}$ for the Kronecker symbol, and $\nu_{p,t}$ for the equivalence Kronecker symbol, meaning $\nu_{p,t} = 1$ iff $p \equiv t$.

Lemma 38. *Let p be a pure term. Then $\theta_p(p) = 1$.*

Proof. Proven by induction on p, direct by definition of the quantity. □

Lemma 39. *Let p be a pure term and t, t' be two terms of the language. If $t \equiv t'$, then $\theta_p(t) = \theta_p(t')$.*

Proof. By induction on each rule defining $\equiv$ in Table 3; the transitive and reflexive case can be derived from this. The first two lines of rules are verified directly as $\mathbb{C}$ is a vectorial space. The three linearity rules also work by definition of summation and the linear constructs. Finally, we can prove the $C_{\equiv}$ case by doing an induction on the depth of $C_{\equiv}$, the base case being just $t \equiv t'$ and then obtaining the result by induction. □

Lemma 40. *Let s, t be two pure terms. Then $\theta_s(t) = \nu_{s,t}$. Furthemore, if s, t are values, then $\theta_s(t) = \delta_{s,t}$.*

Proof. Note by Lemma 38, $\theta_s(s) = 1$. If $\theta_s(t) = 0$, then $s \not\equiv t$, else Lemma 39 would fail. Now, one can check by induction on t that if $\theta_s(t) = 1$, then $s \equiv t$. This is done directly, mainly proving $\equiv$ using $C_{\equiv}$. The second part of the lemma is also obtained directly by induction of t, by looking at the definition of the quantity. □

Lemma 41. *Let t be a term of the language. Suppose $t \not\equiv 0 \cdot t'$. Then t has a canonical form $\sum_{i=1}^{n} \alpha_i \cdot t_i$, that is unique up to reordering and equivalence on the t_i.*

Proof. Let us prove existence by induction on t.

- If $t = x, |0\rangle\,, |1\rangle\,, \lambda x.s, \mathtt{letrec}\, f\, x = s, s_1 s_2$ or $\mathtt{shape}(s)$, then t is directly pure and $t \equiv 1 \cdot t$, which is a canonical form.
- Suppose $t = c(t_1, \ldots, t_n)$. If we have any $t_i \equiv 0 \cdot t_i'$, then $t \equiv 0 \cdot c(t_1, \ldots, t_i', \ldots, t_n)$. Therefore, we can apply the induction hypothesis on each t_i, meaning we have $t_i \equiv \sum_{j_i=1}^{n_i} \alpha_{j_i} \cdot t_{j_i}$. By developing t under $\equiv$, we have $t \equiv \sum_{j_1=1}^{n_1} \cdots \sum_{j_n=1}^{n_n} \alpha_{j_1} \ldots \alpha_{j_n} \cdot c(t_{j_1}, \ldots, t_{j_n})$. This can be seen

as one big sum $\sum_{k=1}^{n_1 \dots n_k} \alpha_k \cdot c(t_1^k, \dots, t_n^k)$, where $\alpha_k = \alpha_{j_1} \dots \alpha_{j_n}$. In particular, $\alpha_k \neq 0$ as each $\alpha_{j_i} \neq 0$; and if $c(t_1^k, \dots, t_n^k) \equiv c(t_1^{k'}, \dots, t_n^{k'})$, then by Lemma 39, $\theta_{c(t_1^k, \dots, t_n^k)}(c(t_1^{k'}, \dots, t_n^{k'})) = 1$ as they are both pure, but by analyzing the formula, this implies $\theta_{t_i^k}(t_i^{k'}) = 1$, thus by Lemma 39, $t_i^k \equiv t_i^{k'}$. By induction hypothesis, this is not possible for all $1 \leq i \leq n$, therefore terms are pairwise not equivalent, which concludes.

- The same process can be done for $t = \texttt{qcase}\, t' \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\}$ or $t = \texttt{match}\, t \left\{ \dots \right\}$, as they have an identical behaviour to a constructor with one slot.

- Finally, suppose $t = \sum_{i=1}^{n} \alpha_i \cdot t_i$. Any $t_i \equiv 0 \cdot t_i'$ will be removed from the sum through $t + 0 \cdot t' \equiv t$, then we can apply the i.h. on the remaining t_i. Finally, using distributivity, we can obtain a big sum $\sum_{k=1}^{n} \beta_k \cdot s_k$, where s_k are pure terms. We can group s_k that are equivalent, then remove any potential term with a 0 phase, and the term obtained satisfies the canonical form definition. Note that as $t \not\equiv 0 \cdot t'$, the sum obtained is actually well defined, in particular there is at least one term.

Unicity comes from our quantity function defined above. Suppose t has two canonical forms $t \equiv \sum_{i=1}^{n} \alpha_i \cdot t_i \equiv \sum_{j=1}^{m} \beta_j \cdot s_j$. As all t_i are pure, for any $1 \leq i_0 \leq n$, $\theta_{t_{i_0}}(\sum_{i=1}^{n} \alpha_i \cdot t_i) = \sum_{i=1}^{n} \alpha_i \theta_{t_{i_0}}(t_i)$. By Lemma 40, $\theta_{t_{i_0}}(t_i) = \nu_{t_{i_0}, t_i}$, however the t_i are not equivalent pairwise, thus $\theta_{t_{i_0}}(\sum_{i=1}^{n} \alpha_i \cdot t_i) = \alpha_{i_0} \neq 0$. However, by the same argument, $\theta_{t_{i_0}}(\sum_{j=1}^{m} \beta_j \cdot s_j) = \sum_{j=1}^{m} \beta_j \theta_{t_{i_0}}(s_j)$. If there is no $s_j \equiv t_{i_0}$, then Lemma 39 fails, thus there is at least one; and there cannot be more than one, else, $s_j \equiv t_{i_0} \equiv s_{j'}$, thus by transitivity $s_j \equiv s_{j'}$, which is not possible as this is a canonical form. Therefore, there exists j_0 such that $\theta_{t_{i_0}}(\sum_{j=1}^{m} \beta_j \cdot s_j) = \beta_{j_0}$; by Lemma 39, we obtain $\alpha_{i_0} = \beta_{j_0}$. Thus, any t_i has a unique and distinct (as they are not equivalent) match in s_j, therefore $n \leq m$. The same process can be done the other way around, so $m \leq n$, thus $m = n$. So each sum contains the same number of elements, such that there is a map from $1, \dots, n$ to $1, \dots, n$ that associates i with j such that $t_i \equiv s_j$ and $\alpha_i = \beta_j$. Therefore, both sums are equal up to an equivalence on t_i and permutation of the t_i. □

Lemma 42. *Let t be a well-typed term, suppose that t terminates and has a canonical form $\sum_{i=1}^{n} \alpha_i \cdot s_i$. Then any element has the same shape, meaning $\texttt{shape}(s_i) \rightsquigarrow^* v_i$, $\texttt{shape}(s_j) \rightsquigarrow^* v_j$, and $v_i = v_j$ for any i, j.*

Proof. By induction on the typing, and the constructed canonical form. Few points to note:

- for (qcase), as $t_0 \perp t_1$, in particular they have the same shape, thus any s_i will reduce to a superposition of t_0, t_1;

- for (match), even if the inside argument is the superposition, by induction hypothesis, all elements have the same shape, thus in particular reduce to the same t_i.

- For (sup), $t_i \equiv t_j$, thus they have the same shape. □

Another process that we will do in proofs is completing canonical forms. Given two terms s, t with canonical forms $\sum_{i=1}^{n} \alpha_i \cdot s_i$ and $\sum_{j=1}^{m} \beta_j \cdot t_j$, one can always create the set S, that contains all the s_i, t_j such that they are pairwise

non equivalent. Then, by adding $0 \cdot t'$, we can write each canonical form as $\sum_{k=1}^{l} \alpha_l \cdot w_l$ and $\sum_{k=1}^{l} \beta_l \cdot w_l$. This is not a canonical form as some phases are zero, but it is equivalent to the first canonical form, thus to s, t; and such shape is actually close enough to keep most existing results on the canonical form, such as normalization.

The following lemma abuses the operational semantics, by supposing that we have a rule allowing to reduce any term $\sum_{i=1}^{n} 0 \cdot t_i$ to $\sum_{i=1}^{n} 0 \cdot t'_i$ when $t_i \rightsquigarrow^? t'_i$. Our type system can easily be expanded to satisfy the properties; in fact, all the following results, i.e., subject reduction and confluence are proven by taking this case into account. Furthermore, we will later prove that any well-typed term cannot be expressed as so, thus this rule is only necessary for the proofs, and we have thus chosen to omit it.

Lemma 43. *Let $t = \sum_{i=1}^{n} \alpha_i \cdot t_i$, suppose $t_i \rightsquigarrow^* v_i$. Then $t \rightsquigarrow^* \sum_{i=1}^{n} \alpha_i \cdot v_i$.*

Proof. For each t_i such that $t_i \equiv 0 \cdot s$, or such that $\alpha_i = 0$, we can remove it via (equiv), thus $t \equiv \sum_{k=1}^{l} \alpha_k \cdot t_k$. By writing each canonical form, up to completion, as $t_k \equiv \sum_{j=1}^{m} \beta_{k,j} \cdot s_j$, $s_j \rightsquigarrow^? s'_j$ and $\sum_{k=1}^{l} \alpha_k \cdot \beta_{k,j} = \gamma_j$:

$$t \equiv \sum_{j=1}^{m} \gamma_j \cdot s_j \equiv \sum_{j,\gamma_j \neq 0} \gamma_j \cdot s_j \rightsquigarrow \sum_{j,\gamma_j \neq 0} \gamma_j \cdot s'_j \equiv \sum_{j=1}^{m} \gamma_j \cdot s'_j \equiv \sum_{k=1}^{n} \alpha_k \cdot t'_k,$$

as the s_j are pairwise not equivalent, either all $\gamma_j = 0$, or it is a canonical form. If the obtained form is a value, then we can stop here, and the result will be correct; even if t_i is not a value, the non-value parts will still be canceled via (Equiv) at the end, as the reduction is deterministic. Else, we can appy (Can), and we can replace $0 \cdot s_j$ by $0 \cdot s'_j$. Furthermore, we can add back the removed terms from the beginning, adding back t'_i instead of t_i, to get that $\sum_{k=1}^{l} \alpha_k \cdot t'_k \equiv \sum_{i=1}^{n} \alpha_i \cdot t'_i$. We conclude by (Equiv), and chaining this multiple times; such process terminates as the t_i do terminate. □

B.2 Proofs of confluence

Lemma 44 (Equivalence preservation)**.** *Let s_1, s_2 be two well-typed terms. Suppose $s_i \rightsquigarrow t_i$ for $i = 1, 2$. Then $t_1 \equiv t_2$.*

Proof. We prove this result by induction of the sum of the depth of the derivation trees of $s_i \rightsquigarrow t_i$.

Consider the base case where both derivation trees have the same root. Apart from (Can) and (Shape_s), having the same root implies that $s_1 = s_2$, and as reduction is deterministic, $t_1 = t_2$, thus $t_1 \equiv t_2$. In the case of (Can), by Lemma 41, if one has a canonical form, then by $\equiv$ the second also has a canonical form, which is identical up to commutation and equivalence on the terms. Thus, $s_1 = \sum_{j=1}^{n} \alpha_i \cdot r_i$, and $s_2 = \sum_{j=1}^{n} \beta_i \cdot q_i$, where both sums are identical, up to commutation and each q_i is equivalent to a r_j. We can apply the induction hypothesis on q_i, r_j, to get that the reduced terms are also equivalent, and up to commuting the terms back, we obtain the equivalence at the end. If not, both terms have 0 phase, and equivalence is direct from this. The same process can be applied for (Shape_s), but inside the `shape` construct, by Lemma 42, as it is a value.

Now, suppose that the trees do not have the same root. First, do not consider any (Shape) rule. Again, apart from (Can) and (Equiv), all left terms are pure, and by Lemma 39, no distinct rule can yield two equivalent terms. Suppose the first tree has a (Equiv) root, meaning:

$$\frac{s_1 \equiv q_1 \qquad q_1 \rightsquigarrow r_1 \qquad r_1 \equiv t_1}{s_1 \rightsquigarrow t_1} \text{ (Equiv)}$$

By transitivity, $q_1 \equiv s_2$, thus we can apply the induction hypothesis on $q_1 \rightsquigarrow r_1$ and $s_2 \rightsquigarrow t_2$ to get $r_1 \equiv t_2$, and by transitivity, $t_1 \equiv t_2$. Same can be done for the right term.

Now, suppose the first tree has a (Can) root. As the right tree has neither a (Can) or (Equiv) root, it has a root with a a pure term. This implies that all phases are not zero, and that s_1 is a canonical form. By uniqueness, $s_1 = 1 \cdot q_1$, with $q_1 \equiv s_2$, and $t_1 = 1 \cdot r_1$. We can apply the induction hypothesis on $q_1 \rightsquigarrow r_1$, $s_2 \rightsquigarrow t_2$, to get $t_1 \equiv t_2$ by transitivity of (Equiv). Same can be done for the right term.

The same process can be done for the (Shape) rules, but inside the `shape` construct. □

Theorem 11 (Confluence)**.** *Given a well-typed term t, if there exist t_1 and t_2 such that $t \rightsquigarrow^* t_1$ and $t \rightsquigarrow^* t_2$, then there exist t_3 and t_4 such that $t_1 \rightsquigarrow^* t_3$, $t_2 \rightsquigarrow^* t_4$ and $t_3 \equiv t_4$.*

Proof. Let us prove semi-confluence, which implies confluence. Let $t \rightsquigarrow t_1$ and $t \rightsquigarrow^* t_2$. If $t = t_2$, then result is direct; else $t \rightsquigarrow t_3 \rightsquigarrow^* t_2$. If $t_3 = t_2$, by the above result, $t_1 \equiv t_2$, which concludes. Else $t \rightsquigarrow t_3 \rightsquigarrow t_4 \rightsquigarrow^* t_2$. By the above result, $t_3 \equiv t_1$. And as $t_3 \rightsquigarrow t_4$, then $t \rightsquigarrow t_4$ by (Equiv). Therefore, $t \rightsquigarrow t_1 \rightsquigarrow t_4 \rightsquigarrow^* t_2$, and we conclude. □

B.3 Proofs for Canonical Forms

Lemma 45. *Let t be a term and s be a pure term. Suppose $\theta_s(t) \neq 0$. Then t has a unique canonical form.*

Proof. Suppose $t \equiv 0 \cdot t'$. Then $\theta_s(t) = \theta_s(0 \cdot t') = 0$ by Lemma 39 and by definition of the quantity; this contradicts the initial hypothesis. We can then conclude by Lemma 41. □

Lemma 12 (Canonical form for typed terms)**.** *Let $\Gamma; \Delta \vdash t : T$ be a well-typed term. Then t has a canonical form $\sum_{i=1}^{n} \alpha_i \cdot p_i$, and this canonical form is unique, up to equivalence on each p_i and term reordering. Furthermore, $\forall\ 1 \leq i \leq n,\ \Gamma; \Delta \vdash p_i : T$.*

Proof. Let us first prove that for any well-typed t, there exists a pure term s such that $\theta_s(t) \neq 0$; we then conclude by Lemma 45. By induction on t:

- If t is pure, then take $s = t$ and result is direct;
- If the root is (cons), meaning $t = c(t_1, \ldots, t_n)$, then by induction hypothesis, as t_i is typed, there is a pure term s_i such that $\theta_{s_i}(t_i) \neq 0$. We can take $s = c(s_1, \ldots, s_n)$ and conclude.
- Same is done for (qcase) and (match).

- If the root is (equiv), $t \equiv t'$, we obtain the result by induction hypothesis on t' and conclude by Lemma 39.
- If the root is (contr), then $t = \sigma(t')$; we can apply the induction hypothesis on t' and take $s = \sigma(s')$ which will preserve σ.
- Finally, if the root is (sup), $t = \sum_{i=1}^{n} \alpha_i \cdot t_i$. By typing, t_i is well-typed, thus by induction hypothesis and Lemma 45, t_i has a canonical form. Furthermore, as $t_i \perp t_j$, each t_i terminates, with $t_i \rightsquigarrow^* \sum_{j=1}^{n_i} \beta_{i,j} \cdot w_j^i$, being canonical forms. One can complement each canonical form to get $t_i \rightsquigarrow^* \sum_{j=1}^{m} \beta_{i,j} w_j$, where the w_j are pure and distinct pairwise. We can thus use Lemma 43, and get that $t \rightsquigarrow^* \sum_{j=1}^{m} (\sum_{i=1}^{n} \alpha_i \beta_{i,j}) \cdot w_j$. If we suppose that $t \equiv 0 \cdot t'$, taking $t' = v$, by Theorem 11, $0 \cdot v \equiv \sum_{j=1}^{m} (\sum_{i=1}^{n} \alpha_i \beta_{i,j}) \cdot w_j$. By Lemma 39, $\theta_{w_j}(0 \cdot v) = 0 = \sum_{i=1}^{n} \alpha_i \beta_{i,j}$. One can check that by definition of $t_i \perp t_j$, the rows $(\beta_{i,1}, \dots, \beta_{i,n})$ are orthogonal, thus this would imply that $\alpha_i = 0$, which contradicts the hypothesis of (sup). Therefore, $t \not\equiv 0 \cdot t'$, by Lemma 41, t has a canonical form $\sum_{k=1}^{l} \gamma_k \cdot u_k$, and take $s = u_1$ to conclude.

Finally, one can remark that by construction of the canonical form in Lemma 41, along with an induction on typing, the pure terms obtained are well-typed; such result is possible as the canonical form is unique, thus the one that we exhibited will match. □

B.4 Proofs of progress

Lemma 46. *Let t be a well-typed pure term of type $B^{|\rangle}$, and v be a value. We suppose that $t \equiv v$, then t is a value.*

Proof. As t is pure, $\theta_t(v) = \theta_t(t) = 1$. By definition of the quantity, it is easy to check that $\theta_t(v) = 1$ implies that t is a value too, thus one can conclude. □

Lemma 47. *Let t be a well-typed term of type $T_1 \rightarrowtail T_2$. Then its canonical form is $1 \cdot t'$.*

Proof. Direct by the way canonical forms are generated in Lemma 41, and by typing of the different constructs. □

Theorem 13 (Progress)**.** *Let $\varnothing; \varnothing \vdash t : T$ be a closed term, either t is equivalent to a value, or t reduces.*

Proof. We prove this result by induction on the typing of t. First, let us consider that t is a pure term.

- The rules (ax), (ax_c) do not derive a closed term.
- The rules (ax_0), (ax_1) yield directly a value.
- If the root is (qcase), then $t = \texttt{qcase}\, s \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\}$, with $\varnothing; \varnothing \vdash s : \text{Qbit}$. Either $s \rightsquigarrow s'$ and $t \rightsquigarrow \texttt{qcase}\, s' \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\}$ through (E); or $s \equiv v$, which by Lemma 46 indicates that s is a pure value, thus $s = |i\rangle$. In any case, we can reduce through (Qcase_i) for $i = 0, 1$.

- Suppose t is typed with (cons), namely $\varnothing;\varnothing \vdash c(t_1,\dots,t_n) : B$. This implies $\varnothing;\varnothing \vdash t_i : B_i$, thus we can apply the induction hypothesis on t_i. If each $t_i \equiv v_i$, then $t \equiv c(v_1,\dots,v_n)$ through $C_{\equiv}$, which is a value. Else, we take the first t_i, starting from the right, such that $t_i \rightsquigarrow t_i'$, and then $t \rightsquigarrow c(t_1,\dots,t_i',\dots,t_n)$ by (E).

- If the root is (match), then $t = \mathtt{match}\, s\, \{c_1(\overrightarrow{x_1}) \to t_1, \dots, c_n(\overrightarrow{x_n}) \to t_n\}$, with $\varnothing;\varnothing \vdash s : B$. Either $s \rightsquigarrow s'$ and $t \rightsquigarrow \mathtt{match}\, s' \left\{\dots\right\}$ through (E); or $s \equiv v$, which by Lemma 46 indicates that s is a pure value, thus $s = \bar{c}(v_1,\dots,v_m)$. As s is of type B, then $\bar{c} \in \mathrm{Cons}(B)$, thus $\bar{c} = c_i$ and $t \rightsquigarrow t_i$ through (Match).

- The rules (abs), (abs_c) and (rec) yield directly a value.

- Suppose the root of the derivation is (app), meaning $t = t_1 t_2$. By typing, t_1, t_2 are also closed terms and the induction hypothesis can be applied. If $t_2 \rightsquigarrow t_2'$, then $t \rightsquigarrow t_1 t_2'$ by (E). Else, $t_2 \equiv v_2$. If $t_1 \rightsquigarrow t_1'$, then $t_1 v_2 \rightsquigarrow t_1' v_2$ by (E), and as $t \equiv t_1 v_2$, then $t \rightsquigarrow t_1' v_2$ by (Equiv). Else, $t_1 \equiv v_1$. By Lemma 47, as t_1 has a functional type, then its canonical form consists of one pure term; as v_1 has the same canonical form by unicity, this term is a value too. Thus, $t_1 \equiv 1 \cdot v \equiv v$. Now, the only closed, well-typed values of type $C \Rightarrow T$ or $T \multimap T'$ are either $\lambda x.t'$, and as $(\lambda x.t')v_2 \rightsquigarrow \sigma(t')$ via (Lbd), then $t \rightsquigarrow \sigma(t')$ via (Equiv); or $\mathtt{letrec}\, f\, x = t'$ and as $(\mathtt{letrec}\, f\, x = t')v_2 \rightsquigarrow \sigma(t')$ via (Rec), then $t \rightsquigarrow \sigma(t')$ via (Equiv).

- The rule (sup) does not derive a pure term.

- Suppose the root is (shape), meaning $t = \mathtt{shape}(s)$, with $\varnothing;\varnothing \vdash s : B^{|\rangle}$. Note that s is actually closed because its linear and non-linear context are joined in the non-linear context of t which is empty. Then either $s \rightsquigarrow s'$ and $t \rightsquigarrow \mathtt{shape}(s')$ via (E); or $s \equiv v$. Either $v = |i\rangle$, and t reduces through (Shape_i) and (Equiv); or $v = c(v_1,\dots,v_n)$ and t reduces through (Shape_c) and (Equiv); or v is a summation, as it is well-typed it has a canonical form w, and by transitivity $t \equiv w$, thus t reduces through (Shape_s) and (Equiv). Any other value cannot be typed with a type $B^{|\rangle}$.

- Suppose the root of the derivation is (equiv), namely we have $t' \equiv t$. By hypothesis, t is closed and thus t' is too. Therefore, either $t' \equiv v$ and by transitivity of $\equiv$, $t \equiv v$; else $t' \rightsquigarrow t''$ and then $t \rightsquigarrow t''$ by (Equiv).

- The rule (contr) does not derive a closed term.

- Finally, the rule (contr) cannot yield a closed term.

Now, let t be a well-typed closed term; let us consider its canonical form $t' = \sum_{i=1}^{n} \alpha_i \cdot t_i$. If t' is a value, then t is equivalent to a value, and we conclude. Else, any term t_i verifies $t_i \rightsquigarrow^{?} t_i'$ for a given t_i', by definition, and thus $t' \rightsquigarrow \sum_{i=1}^{n} \alpha_i \cdot t_i'$ by (Can), and thus $t \rightsquigarrow \sum_{i=1}^{n} \alpha_i \cdot t_i'$ by (Equiv), which concludes. □

B.5 Proofs for Subject Reduction

Lemma 48 (Substitution lemma)**.** *The two following properties hold:*
- If $\Gamma, x : T'; \Delta \vdash t : T$*,* $\Gamma; \varnothing \vdash t' : T'$ *and* $\sigma = \{x \to t'\}$*, then* $\Gamma; \Delta \vdash \sigma(t) : T$*.*
- If $\Gamma; \Delta, x : T' \vdash t : T$*,* $\Gamma; \Delta' \vdash t' : T'$ *and* $\sigma = \{x \to t'\}$*, then* $\Gamma; \Delta, \Delta' \vdash \sigma(t) : T$*.*

Proof. We prove this by induction on the typing tree of t. For the first property:

- Suppose the root is (ax), meaning $\Gamma, x : T'; y : T \vdash y : T$, then one can check that we can also type directly $\Gamma; y : T \vdash y : T$, and by definition $\sigma(y) = y$ as $y \notin \mathrm{Supp}(\sigma)$, which concludes.
- Suppose the root is (ax_c), then either $t = y$ with $y \neq x$, and the same process can be done as above. Else, we have $\sigma(t) = \sigma(x) = t'$, which concludes by the typing of t'.
- If the rules are (ax_0) or (ax_1), then again $\sigma(t) = t$ and we can derive the typing of t with Γ instead of $\Gamma, x : T'$.
- Suppose the root is (qcase), thus $t = \mathtt{qcase}\, t' \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\}$. By induction hypothesis, x is removed from the context of t', t_0, t_1, thus we conclude by typing $\sigma(t) = \mathtt{qcase}\, \sigma(t') \left\{|0\rangle \to \sigma(t_0)\,, |1\rangle \to \sigma(t_1)\right\}$.
- Suppose the rule is (cons), meaning $\Gamma, x : T'; \Delta_i \vdash t_i : A_i$; by i.h. $\Gamma; \Delta_i \vdash \sigma(t_i) : A_i$, and thus we can type $\Gamma; \Delta \vdash c(\sigma(t_1), \dots, \sigma(t_n)) : B$, which concludes.
- Suppose the rule is (match), the same process can be done as for (qcase), by remarking that x cannot be a bound variable of a t_i, else typing could not happen.
- Suppose the root is (abs), thus we have $\Gamma, y : T''; \Delta, x : T \vdash t : T'$; by i.h. $\Gamma; \Delta, x : T \vdash \sigma(t) : T'$, thus by (abs) we can type $\Gamma; \Delta \vdash \lambda x.\sigma(t) : T \multimap T'$, which concludes as $\lambda x.\sigma(t) = \sigma(\lambda x.t)$ as $x \notin \mathrm{Supp}(\sigma)$.
- The same process can be done for (abs_c): we have $\Gamma, y : T'; \Delta \vdash \lambda x.t : C \Rightarrow T$, thus we can apply the induction hypothesis to $\Gamma, x : C, y : T'; \Delta \vdash t : T$ as $x \neq y$, which gives $\Gamma, x : C; \Delta \vdash \sigma(t) : T$, and we can conclude.
- Suppose the root is (rec), then the same process as for (abs_c) can be applied.
- Suppose the root is (app), meaning we have $\Gamma, x : {B^{|\rangle}}''; \Delta \vdash t_1 : B^{|\rangle} \multimap {B^{|\rangle}}'$ and $\Gamma, x : {B^{|\rangle}}''; \Delta' \vdash t_2 : B^{|\rangle}$; by i.h. we have $\Gamma; \Delta \vdash \sigma(t_1) : B^{|\rangle} \multimap {B^{|\rangle}}'$ and $\Gamma; \Delta' \vdash \sigma(t_2) : B^{|\rangle}$, and thus by (app) we can type $\Gamma; \Delta \vdash \sigma(t_1)\sigma(t_2) : {B^{|\rangle}}'$ which concludes.
- The same can be done for (app_c).
- Suppose the root is (sup), meaning we have $\Gamma, x : T'; \Delta \vdash t_i : T$. By induction hypothesis, we get $\Gamma; \Delta \vdash \sigma(t_i) : T$, and by definition of $\perp$, $t_i \perp t_j$ implies $\sigma(t_i) \perp \sigma(t_j)$, and thus we can type by (sup) $\Gamma; \Delta \vdash \sum_{i=1}^{n} \alpha_i \cdot \sigma(t_i) : T$, which concludes by the action of σ on superpositions.

- Suppose the root of the derivation is (shape), first remark that x belongs to the Γ part, because it has no marker around it; thus we have $\Gamma, x : T'; \Delta \vdash t : B^{|\rangle}$, thus by i.h. $\Gamma; \Delta \vdash \sigma(t) : B^{|\rangle}$, and we can type by (shape) $\Gamma, [\Delta]; \varnothing \vdash \texttt{shape}(\sigma(t)) : \texttt{shape}(B^{|\rangle})$, which concludes.
- Suppose the root is (contr), meaning we have $\Gamma, z : T', [x : B^{|\rangle}]; \Delta, y : B^{|\rangle} \vdash t : T$. Again, as z comes with no marker and thus $z \neq x$, which means that we can treat the case as before, with a non-linear context $\Gamma, [x : B^{|\rangle}]$, and conclude.
- Suppose the root is (equiv), thus $\Gamma, x : T'; \Delta \vdash t : T$; by i.h. $\Gamma; \Delta \vdash \sigma(t) : T$. One can check that if $t \equiv t'$, then $\sigma(t) \equiv \sigma(t')$, which concludes.

A similar process can be done to prove the second property. It differs only by the fact that x is now present in only one subterm context; for the other subterm, while some x may be present by a `shape` construct, replacing it with a well-typed value of same type, one can rewrite the typing tree with the new term (possibly by remodeling the (contr) rules), and conclude. Else, we have directly $\sigma(t) = t$, thus typing is direct. □

Corollary 49. *Let $\Gamma, x_1 : T_1, \dots, x_m : T_m; \Delta, x_{m+1} : T_{m+1}, \dots, x_n : T_n \vdash t : T$ be a well-typed term. Then, for any substitution σ such that for all $1 \leq i n$, $x_i \in \mathrm{Supp}(\sigma)$, $\Gamma; \Delta_i \vdash \sigma(x_i) : T_i$, we have $\Gamma; \Delta, \Delta_1, \dots, \Delta_n \vdash \sigma(t) : T$.*

Proof. Direct by induction on n: the case $n = 1$ is simply the above lemma, and for $n = k + 1$, any substitution σ can be written as $\sigma = \sigma' \circ \tau$ with $\tau = \{x_1 \to \sigma(x_1)\}$; $\tau(t)$ is typable by the above lemma and removes $x_1 : A_1$ from the context, and then $\sigma'(\tau(t))$ concludes by induction hypothesis. □

Definition 50. *Let $\Gamma; \Delta \vdash t : T$ be a well-typed term. We say that t types and terminates, written t.a.t., if $t \rightsquigarrow t_1 \dots \rightsquigarrow t_k \rightsquigarrow v$, $\Gamma; \Delta \vdash t_i : T$, $\Gamma; \Delta \vdash v : T$.*

Lemma 51. *Let $(t_i)_{1 \leq i \leq n}$ be well-typed terms such that $\forall i \neq j, t_i \perp t_j$. Let $s = \sum_{i=1}^n \alpha_i \cdot t_i$, and $t = \sum_{i=1}^n \beta_i \cdot t_i$ such that s, t are t.a.t. and $\sum_{i=1}^n \alpha_i \beta_i^* = 0$. Then $s \perp t$.*

Proof. As $t_i \perp t_j$, each t_i reduces to a canonical form. Let us write them up to completion as $t_i \rightsquigarrow^* \sum_{j=1}^m \gamma_{ij} \cdot v_j$. By Lemma 43, $s \rightsquigarrow^* \sum_{j=1}^m (\sum_{i=1}^n \alpha_i \gamma_{ij}) \cdot v_j$, and $t \rightsquigarrow^* \sum_{j=1}^m (\sum_{i=1}^n \beta_i \gamma_{ij}) \cdot v_j$. By definition, this is a quasi canonical form, as some phases may be zero, but v_j are two by two distinct; proving the orthogonality condition on this will imply orthogonality for the exact canonical form. First, it is easy to see that they all have the same shape, as t_i are orthogonal, thus have the same shape. Compiling the inner product gives the following:

$$\sum_{j=1}^m \sum_{j'=1}^m (\sum_{i=1}^n \alpha_i \gamma_{ij})(\sum_{i=1}^n \beta_i \gamma_{ij'}) \delta_{v_j, v'_j} = \sum_{1 \leq i, i' \leq n} \alpha_i \beta_i^* \sum_{j=1}^m \gamma_{ij} \gamma_{ij'}^* = \sum_{1 \leq i, i' \leq n} \alpha_i \beta_i^* \delta_{i,i'} = 0$$

where the second equality is obtained by developing each sum, and the third by definition of $t_i \perp t_j$, and we conclude by the condition on α_i, β_i. □

Lemma 52. *Let $(t_i)_{1 \leq i \leq n}$, $(t'_i)_{1 \leq i \leq n}$ such that $\forall ij, t_i \perp t'_j$. Then for any α_i, β_i, $\sum_{i=1}^n \alpha_i \cdot t_i \perp \sum_{i=1}^n \beta_i \cdot t'_i$.*

Proof. Consider $(\tilde{t_i})_{1\leq i\leq 2n}$ where $\tilde{t_i} = t_i$ for $1 \leq i \leq n$, and $\tilde{t_i} = t'_i$ for $n < i \leq 2n$, and apply Lemma 51. □

Lemma 53. *Let $\Gamma; \Delta \vdash t : T$ be a pure term. Then we have the following typing tree:*

$$\dfrac{\dfrac{\dfrac{}{\Gamma_0; \Delta \vdash s : T}\ (r)}{\Gamma_1; \Delta \vdash \sigma_1(s) : T}\ (contr)}{\begin{array}{c}\vdots\\ \Gamma_n; \Delta \vdash \sigma_n(s) : T\end{array}}\ (contr)$$

where $n \geq 0$, each $\Gamma_n = \Gamma$, $\sigma_n(s) = t$, r is neither (contr) nor (equiv), and each Γ_i, respectively σ_i, contains one less marked variable, contains one more mapping, as per (contr).

Proof. Let s be a pure term, and $\Gamma; \Delta \vdash t : T$ be any term. We prove by induction on typing of t that if $\theta_s(t) \neq 0$, then s satisfies the property of the lemma.

- (ax): $t = x$, $\theta_s(t) \neq 0 \implies s = x$, and we can type s directly.
- (ax_c), (ax_0) and (ax_1) are obtained similarly.
- (qcase): $t = \texttt{qcase}\, r \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\}$, $\theta_s(t) \neq 0 \implies s = \texttt{qcase}\, q \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\} \wedge r \equiv q$ by Lemma 40 as q is pure. We can type q via (equiv) by typing r first, and then type s via (qcase).
- (cons): $t = c(t_1, \ldots, t_n)$, non-zero quantity implies $s = c(s_1, \ldots, s_n)$ with $s_i \equiv t_i$. We can type t_i by the typing of t, then s_i via (equiv), then s via (cons).
- (match): same as for (qcase).
- (abs), (abs_c), (rec): same as (ax).
- (app): this implies $t = t_1 t_2$, $s = s_1 s_2$, with $s_i \equiv t_i$, thus we type s_i through (equiv) and conclude.
- (app_c): same as (app).
- (sup): $t = \sum_{i=1}^{n} \alpha_i \cdot t_i$, $\theta_s(t) = \sum_{i=1}^{n} \alpha_i \theta_s(t_i)$. As $\theta_s(t) \neq 0$, then there is $1 \leq i \leq n$ such that $\theta_s(t_i) \neq 0$. We can apply the induction hypothesis on t_i, as it has the same context and type as t.
- (shape): $t = \texttt{shape}(r)$, $s = \texttt{shape}(q)$ and $q \equiv r$, thus we type q through (equiv) and conclude.
- (equiv): $t \equiv t'$, $\theta_s(t) = \theta_s(t)' \neq 0$ through Lemma 39, and we apply i.h. on t' as it has the same context and type.
- (contr): $t = \sigma(r)$. We can apply the induction hypothesis on r, then reapply (contr) on the alternative tree.

Now, we can conclude as by Lemma 38, $\theta_s(s) = 1$; and any obtained tree satisfies the hypothesis. □

Lemma 54. *Let* $\varnothing; x : B^{|\rangle} \vdash s : B^{|\rangle\prime}$, $\sigma_v = \{x \to v\}$, $\varnothing;\varnothing \vdash v : B^{|\rangle}$, $\varnothing;\varnothing \vdash v' : B^{|\rangle}$ *such that* $v \perp v'$ *and* s *t.a.t.* . *We also suppose that any term* t *that terminates in less steps than* s *is also t.a.t.* . *Then* $\sigma_v(s) \perp \sigma_{v'}(s)$.

Something that we can note is that given $s \rightsquigarrow^* \alpha \cdot |0\rangle + \beta \cdot |1\rangle$, then $\texttt{qcase}\, s \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\}$ will not always reduce to $\alpha \cdot t_0 + \beta \cdot t_1$, as $(C_\equiv)$ requires a pure term, which is not guaranteed for s. However, we will have $\texttt{qcase}\, s \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\} \rightsquigarrow^* \alpha \cdot s_0 + \beta \cdot s_1$, where $t_i \rightsquigarrow^* s_i$. In particular, proving results of orthogonality between such terms will guarantee orthogonality between the t_i. Same goes for `match`.

Proof. By induction on the typing of s:

- (ax) gives the desired result directly;
- (ax_c), (ax_0) and (ax_1) have an empty linear context;
- (qcase): $s = \texttt{qcase}\, t \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\}$; as s terminates, so does t, in less steps than s, thus t reduces to a well-typed value by hypothesis. Therefore, either $\varnothing; x : B^{|\rangle} \vdash t : \text{Qbit}$; the only pure closed values of type Qbit are $|0\rangle$ and $|1\rangle$, thus $\sigma_v(t) \rightsquigarrow^* \alpha \cdot |0\rangle + \beta \cdot |1\rangle$, $\sigma_{v'}(t) \rightsquigarrow^* \gamma \cdot |0\rangle + \delta \cdot |1\rangle$, and by orthogonality $\alpha\gamma^* + \beta\delta^* = 0$. By definition, $\sigma_v(s) \rightsquigarrow^* \alpha \cdot t_0 + \beta \cdot t_1$, $\sigma_{v'}(s) \rightsquigarrow^* \gamma \cdot t_0 + \delta \cdot t_1$, which are well-typed as s is t.a.t., and we can conclude by Lemma 51. Else, $\sigma_v(t) = t$, thus $t \rightsquigarrow^* \alpha \cdot |0\rangle + \beta \cdot |1\rangle$, $\sigma_v(s) \rightsquigarrow^* \alpha \cdot \sigma_v(t_0) + \beta \cdot \sigma_v(t_1)$. In particular, $t_0 \perp t_1$, thus $\sigma_v(t_i) \perp \sigma_{v'}(t_{1-i})$, and $\sigma_v(t_i) \perp \sigma_{v'}(t_i)$ by induction hypothesis, thus we conclude by Lemma 52, as the obtained summation is t.a.t. as a reduced of s.
- (cons): there is $1 \leq i \leq n$ with $\varnothing; x : B^{|\rangle} \vdash t_i : B^{|\rangle}{}_i$. As one coordinate is orthogonal and the others are identical, orthogonality is verified for the whole term.
- (match): same as for (qcase). The only difference is that $\sigma(t)$ will reduce to a linear combination of same constructor, thus $\sigma_v(s) \rightsquigarrow^* \sum_{j=1}^m \beta_j \cdot \sigma_{w_j}(t_i)$, where $w_j \perp w'_j$, and the result is direct by induction hypothesis and by concluding with Lemma 52.
- (abs), (abs_c), (rec) do not yield a term of type $B^{|\rangle}$.
- (app): by typing, reduction of $t_1 t_2$ implies that t_1 reduces too to a well-typed closed value, i.e., either $\lambda x.t$ or `letrec`. Furthermore, by typing, either x is in the context of t_1, thus of t, or in the context of t_2, thus of v. In any case, by joining all substitutions, we reach a point where $s \rightsquigarrow^* \tilde{\sigma}(t)$, and we can apply the i.h. on t, as it is t.a.t. as a reduced from s. Same goes for (app_c).
- (sup): all t_i have the x in the context, thus $\sigma_v(t_i) \perp \sigma_{v'}(t'_i)$, and $\sigma_v(t_i) \perp \sigma_{v'}(t_j)$ for $i \neq j$ as $t_i \perp t_j$, thus we conclude by Lemma 52, as we apply the lemma on s, which is t.a.t. itself.
- (shape): no linear context.
- (equiv): apply the hypothesis on t, then via (Equiv) we can obtain a reduction for s.

- (contr): just an alpha-renaming of a classical variable, does not disturb the previous proofs. □

Lemma 55 (Strong Subject Reduction)**.** *Let $\Gamma;\Delta \vdash t : T$ be a well-typed term, such that t terminates in k steps. Then t t.a.t. .*

Proof. This is proven by induction on the depth of termination of t. Let us first prove that t reduces to well-typed terms. If $k = 0$, the result is direct; suppose $k = k' + 1$, thus $t \rightsquigarrow t' \rightsquigarrow^{\leq k'} v$. We will prove that t t.a.t. by induction on the derivation of $\rightsquigarrow$. In most cases, we obtain that t' is well-typed, which implies directly that t t.a.t. by induction hypothesis on t'.

First, note that apart from (Can) and (Equiv), left terms are pure, thus we can apply Lemma 53 to gain a term with no (equiv) rule. Now, let us first suppose that there is no (contr) rule near the end of the typing tree.

- (Qcase_0): the only non (contr) and (equiv) rule typing this syntax is (qcase), thus $\Delta = \Delta_1, \Delta_2$ with $\Gamma;\Delta_1 \vdash |0\rangle : \text{Qbit}$ and $\Gamma;\Delta_2 \vdash t_0 : Q$. Again, as $|0\rangle$ is pure, the only no (contr) and (equiv) rule typing $|0\rangle$ is (ax_0). Therefore, $\Delta_1 = \emptyset$, and we can conclude by typing of t_0.
- (Qcase_1): same as above.
- (Match): the root of typing is (match), thus $\Delta = \Delta_1, \Delta_2$, with $\Gamma;\Delta_1 \vdash c_i^{s_i}(p_v^1, \dots, p_v^{n_i}) : B$ and $\Gamma, \overrightarrow{y_i} : \overrightarrow{C_i}; \Delta_2, \overrightarrow{z_i} : \overrightarrow{Q_i} \vdash t_i : B^{|\rangle}$. Again, by typing as the v_i are pure, this gives us $s_i = B_1^{|\rangle}, \dots, B_n^{|\rangle} \to B$, and $\Gamma;\Delta_i^c \vdash p_v^i : B_i^{|\rangle}$, with $\Delta_1 = \Delta_1^c, \dots, \Delta_{n_i}^c$. As $\overrightarrow{x_i} = \overrightarrow{y_i}, \overrightarrow{z_i}$, we conclude using Corollary 49.
- (Lbd) : If the root is (app), then $\Delta = \Delta_1, \Delta_2$, $\Gamma;\Delta_1 \vdash \lambda x.s : T' \multimap T$ and $\Gamma;\Delta_2 \vdash v : T'$. Typing of $\lambda x.s$ implies $\Gamma;\Delta_1, x : T' \vdash s : T'$, and thus by substitution lemma on $\sigma(s)$ we conclude. The same process can be done if the root is (app_c).
- (Rec) is identical to (Lbd), up to some small tweaks.
- (Can) : we consider an alternative typing tree where all equivalence relations are at the end, as we make them commute with summation, `qcase`, `match` and constructors. Therefore, we have $t \equiv s$, where s has no (equiv) rule near the end. If s is pure, then $t = 1 \cdot s$ (by quantity), by i.h. this reduces to $1 \cdot s'$ where s' is well-typed, and using (equiv) we can type $1 \cdot s'$. Else, s is not pure. Let us first prove that given two well-typed closed terms s, t such that $s \perp t$ and that they terminate, reducing to well-typed closed value, then developing linearly preserves the orthogonality, meaning:
  - $\texttt{qcase}\, s \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\} \perp \texttt{qcase}\, t \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\}$: we have $s \rightsquigarrow^* v$ and $t \rightsquigarrow^* w$ (they terminate as we can derive orthogonality). By definition, v, w are well-typed closed values, thus $v \equiv \alpha \cdot |0\rangle + \beta \cdot |1\rangle$ and $w \equiv \gamma \cdot |0\rangle + \delta \cdot |1\rangle$. As they are orthogonal, then they will verify $\alpha\gamma^* + \beta\delta^* = 0$ (even if this is not the canonical form as some coefficients can be zero, this will still hold). Now, this indicates, by E, that $\texttt{qcase}\, s \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\} \rightsquigarrow^* \alpha \cdot s_0 + \beta \cdot s_1$

and $\texttt{qcase}\, t \left\{ |0\rangle \to t_0 \,, |1\rangle \to t_1 \right\} \rightsquigarrow^* \gamma \cdot s_0 + \delta \cdot s_1$, where $t_0 \rightsquigarrow^* s_0$ and $t_1 \rightsquigarrow^* s_1$. By Lemma 51, as $t_0 \perp t_1$, then $s_0 \perp s_1$, and thus $\alpha \cdot s_0 + \beta \cdot s_1 \perp \gamma \cdot s_0 + \delta \cdot s_1$.

- $c(s, \dots, t_n) \perp c(t, \dots, t_n)$: we suppose that s, t are in first slot, other cases are done equivalently. By E, when reducing each term to a value, the first slot will contain the reduced value of s and t. Then, as they are orthogonal, and the other values are identical, they have the same shape by Lemma 42, and when writing the summation to test the zero equality, we can just develop through the first coordinate to obtain back the summation from the orthogonality of s and t.
- $\texttt{match}\, s \left\{ \dots \right\} \perp \texttt{match}\, t \left\{ \dots \right\}$: by definition, $s \rightsquigarrow^* \sum_{i=1}^n \alpha_i \cdot w_i$ and $t \rightsquigarrow^* \sum_{i=1}^n \beta_i \cdot w_i$, by writing their canonical form, and then completing by some $0 \cdot w_i$ in order to have the same pure elements on each side. As they are orthogonal, then $\sum_{i=1}^n \alpha_i \beta_i^* = 0$. Furthermore, they must have the same shape, and be a value of type B, thus any pure value satisfies $w_i = c_k(v_i^1, \dots, v_i^n)$. As $w_i \neq w_j$ for $i \neq j$, then one coordinate is distinct. Now, $\texttt{match}\, s \left\{ \dots \right\} \rightsquigarrow^* \sum_{i=1}^n \alpha_i \cdot s_i$ and $\texttt{match}\, t \left\{ \dots \right\} \rightsquigarrow^* \sum_{i=1}^n \beta_i \cdot t_i$, where $\sigma(w_i) \rightsquigarrow^* s_i$, and $\sigma(w_i) \rightsquigarrow^* t_i$. For any $i \neq j$, σ_i and σ_j have map one variable to a distinct pure value, as explained above; and any $v \neq w$ where v, w are pure values satisfy $v \perp w$. We can thus apply Lemma 54, and get $\sigma_i(w_i) \perp \sigma_j(w_j)$. Finally, we can conclude by Lemma 51, and as s_i, t_i are reduced terms from $\sigma_i(w_i)$, orthogonality is preserved.

Now, as s is not pure, then there is a subterm of s that contains a (sup) typing rule, followed by some (maybe 0) linear rules, i.e., (cons), (match) or (qcase), as they are the only rules which need a pure term inside their construction to be pure. By definition of $C_{\equiv}$, we can develop linearly this sum to obtain $s' = \sum_{i=1}^n \alpha_i \cdot t_i$; and by construction, we can use the above result by induction, as each term is a subterm of a terminating term, thus reduces in less steps than the original term, therefore we can apply the induction hypothesis. Note that some terms may obtain a phase 0 at the end, and could have reduced in more steps; but as they will be removed, not applying this process is ok, as we will not test orthogonality with them anyway. as the base case is (sup) thus requires orthogonal terms. Therefore, by syntax, s' is a sum of orthogonal well-typed terms. We can reapply this process on each t_i until they are pure, which terminates as t_i has a smaller typing tree than s, as one (sup) rule has been removed. At the end, we obtain multiple summation of pure orthogonal values $\sum_{i=1}^n \alpha_i \cdot (\cdots \sum_{k=1}^l \beta_k \cdot t_k)$. Each sum can be typed through (sup), as the coefficients come from a (sup) rule and the above cases show that orthogonality is kept. We can then develop everything, group equivalent pure terms, and obtain the canonical form back. However, after reducing each term, we can do the reverse and rewrite the obtained term as multiple summations. Furthermore, each term has either reduced, or stayed the same as it was removed; by induction hypothesis on $t_i \rightsquigarrow^? t_i'$, t_i' is well-typed with the same context and type as t; and the orthogonality conditions are still true as we reduce terms, thus we can still type (sup) for each sum, and therefore type the big sum; as this sum is equivalent to $\sum_{i=1}^n \alpha_i \cdot t_i'$, by (equiv), we

can type it and conclude.

- (Shape_0) : by typing, we have $\Gamma'; \Delta' \vdash |0\rangle : T'$ where $\Gamma = \Gamma, [\Delta'], \Delta = \emptyset$ and $T = \texttt{shape}(T')$. Again, as $|0\rangle$ is pure, then we need to type it with (ax_0), which gives $\Delta' = \emptyset$ and $T = \texttt{shape}(\text{Qbit}) = \mathbb{1}$. We conclude as we can type $\Gamma; \varnothing \vdash () : \mathbb{1}$ through (cons).
- (Shape_1) : same as above.
- (Shape_c) : as the root is (shape), typing gives $\Gamma = \Gamma', [\Delta']$, $\Delta = \emptyset$, $T = [B]$ and $\Gamma; \Delta \vdash c(v_1, \dots, v_n) : B$. Again by typing, the root being (cons), we have $\Gamma; \Delta_i \vdash v_i : B_i^{|\rangle}$ and $\Delta = \Delta_1, \dots, \Delta_n$. Therefore, (shape) gives $\Gamma, [\Delta_i]; \varnothing \vdash \texttt{shape}(v_i) : \texttt{shape}(B_i^{|\rangle})$; as $[\Delta] = [\Delta_1], \dots, [\Delta_n]$, the definition of $\tilde{c}$ concludes.
- (Shape_s): by Lemma 12, each v_i is well-typed with the same type as the summation, and we conclude as above.
- (Equiv): By (equiv), $\Gamma; \Delta \vdash t_1 : T$, by i.h., $\Gamma; \Delta \vdash t_1' : T$, and by (equiv) again on $t_1 \equiv t'$, we conclude.
- (E): by i.h. on $t \rightsquigarrow t'$, $\Gamma; \Delta \vdash t' : T$, thus one can replace the typing tree of t by the typing tree of t', and any rule will still hold. This is in particular true because E is made of no superpositions, so none of these checks happen and thus typing is preserved.

Now, given a term with some (contr) rules, we may consider the following:

- One can first remark that any (contr) rule commutes with a (qcase), (match), or any (app) rule. Therefore, for the (Qcase), (Match), (Lbd), (Rec) rules, we can consider an alternative typing tree for t with $k \geq 0$ (contr) rules at the end, and thus have $t = \sigma_1 \dots \sigma_k(s)$, where s has no (contr) rule near the end, thus we can apply the hypothesis above. Also note that if $t \rightsquigarrow t'$, then $t' = \sigma_1 \dots \sigma_k(s')$ and $s \rightsquigarrow s'$. We can then apply the result above on $s \rightsquigarrow s'$, and reapply the k (contr) rules on s' to obtain the well-typedness of t'.
- The (Can) rule contains terms that all have the same context, thus any (contr) rule may actually be applied on each t_i before.
- Any (Shape) rule contains only values, thus no marked variable is actually given by the context and no (contr) rule is needed.

Therefore, the result still holds for any term. □

Theorem 14 (Subject reduction)**.** *Let $\Gamma; \Delta \vdash t : T$ be a well-typed term, and $t \rightsquigarrow t'$. If t terminates or t is pure, then $\Gamma; \Delta \vdash t' : T$.*

Proof. If t terminates, then we can apply Lemma 55. If t is pure, one can remark that the proof in Lemma 55 for pure terms does not require termination, as (Can) is stopped from going further and needing the termination when t is pure, thus one can adapt the proof to get the result. □

C Proofs for Section 3.2

Theorem 16 (Undecidability of orthogonality)**.** *Deciding orthogonality between two well-typed terms is* Π_2^0*-complete.*

Proof. Let us first prove the Π_2^0-hardness. Let us recall the definition of Programming Computable Functions (PCF), which is defined with the following types and terms grammar:

$$A, B ::= \texttt{nat} \mid A \to B$$
$$t, t_1, t_2 ::= x \mid \lambda x.t \mid t_1 t_2 \mid \texttt{fix}\ t \mid \texttt{n} \mid \texttt{succ} \mid \texttt{pred} \mid \texttt{ifz}\ t_1 t_2$$

Each term has a corresponding typing rule, and thus PCF is a typed language. It is easy to see that `Hyrql` can encode PCF: all of PCF's constructions are already present in `Hyrql`, apart from the following terms:

$$\texttt{fix}\ t = \texttt{letrec}\ f\ x = t\ f\ x$$
$$\texttt{pred} = \lambda x.\texttt{match}\ x\ \{0 \to 0, S\ n \to n\}$$
$$\texttt{ifz} = \lambda x.\lambda y.\lambda n.\texttt{match}\ n\ \{0 \to x, S\ m \to y\}$$

Therefore, any term of PCF can be encoded into a term in `Hyrql` through a total computable function. The Universal Halt problem on PCF is known to be Π_2^0-hard. We define `UHalt` as the set of terms of PCF, thus terms in our language, terminating over any input, meaning $tt_1 \dots t_n$ terminates.

Given a well-typed term t from PCF of type $T_1 \rightarrowtail \dots \rightarrowtail T_n \rightarrowtail B^{|\rangle}$ (such type can always be obtained by the Lemma 36), we define

$$g(t) = (tx_1 \dots x_n, |0\rangle), (tx_1 \dots x_n, |1\rangle),$$

where $x_1, \dots, x_n$ are neither free nor bound variables of t. If $t \in \texttt{UHalt}$, then it terminates over any input. In particular, this implies that $tx_1 \dots x_n$ terminates for any substitution, reducing to a value v; and as $(v, |0\rangle) \perp (v, |1\rangle)$, $g(t) \in \texttt{ORTHO}$. If $g(t) \in \texttt{ORTHO}$, then t must terminate for any substitution of $x_1, \dots, x_n$, thus terminates for any value. Therefore, $t \in \texttt{UHalt}$. Thus, $t \in \texttt{UHalt} \iff g(t) \in \texttt{ORTHO}$, $\texttt{UHalt} \leq \texttt{ORTHO}$, and as `UHalt` is Π_2^0-hard, it concludes.

The fact that $\texttt{ORTHO} \in \Pi_2^0$ is direct by definition. Given t, t' be two terms of the same type with $\mathrm{FV}(t) = \mathrm{FV}(t') = \{x_1, \dots, x_n\}$, and let $\sigma_{v_1,\dots,v_n} = \{x_i \to v_i\}_{1 \leq i \leq n}$, then orthogonality decision can be written up as:

$$\forall v_1, \dots \forall v_n, \exists k \in \mathbb{N},\ \sigma_{v_1,\dots,v_n}(\texttt{shape}(t)) \rightsquigarrow^{\leq k} v_s, \sigma_{v_1,\dots,v_n}(\texttt{shape}(t')) \rightsquigarrow^{\leq k} v'_s$$
$$\sigma(t) \rightsquigarrow^* \sum_{i=1}^{n} \alpha_i \cdot v_i \in \texttt{CAN}, \sigma(t') \rightsquigarrow^* \sum_{j=1}^{m} \beta_j \cdot w_j \in \texttt{CAN}$$
$$v_s = v'_s \wedge \sum_{i=1}^{n} \sum_{j=1}^{m} \alpha_i \beta_j^* \delta_{v_i, w_j} = 0$$

The inside property is decidable: checking if a term reducing in less than k steps to a variable, written $t \rightsquigarrow^{\leq k} v$, is decidable; checking the first equality is decidable as there is no superposition (computed in $\mathcal{O}(n)$ with n the size of v), and the equality to 0 of the double sum and equality is decidable as we have restricted the scalars in $\bar{\mathbb{Q}}$. Thus the inside property is Π_0^0; by definition, `ORTHO` belongs to Π_2^0, and thus is Π_2^0-complete. □

Proposition 17. *Given a finite type $B^{|\rangle}$, and two terminating terms $\varnothing;\varnothing \vdash s : B^{|\rangle}$, $\varnothing;\varnothing \vdash t : B^{|\rangle}$, it is decidable whether they are orthogonal.*

Proof. One can check that any value of type T is of size $|v| \leq |T|_d$, by definition of the depth, therefore we use n as a bound on the size of any value of such type. In order to compute and decide `ORTHO`, we first need to reduce s and t to values, which is done in f, g steps. Then, computing $\texttt{shape}(s)$ and $\texttt{shape}(t)$ is done linearly in the size of the value, same for computing equality, thus this is done in $\mathcal{O}(n)$ steps. For the computation of the equality, one needs to compute the canonical forms for the reduced values of s and t. In particular, we can complement each canonical form, so that testing δ_{v_i,w_j} is done directly by comparing the index. This implies checking all terms of the canonical form, and comparing it with all the other terms, thus checking $\mathcal{O}(n)$, comparing it with $\mathcal{O}(n)$ terms, and comparison is done in the worst case in $\mathcal{O}(n)$ steps. Then, we compute each $\alpha_i\beta_j^*$, and then we have to sum n terms and checking nullity. As we have assumed to reduce the set of complex phases such that this can be done polynomially, we assume it is not the operation with the most cost. Such calculus needs to be done in the worst case n^3 times. □

D Proofs for Section 3.3

Theorem 25 (Bounded circuit compilation)**.** *Let $T : \mathbb{N} \to \mathbb{N}$, and let $t \in \texttt{Hyrql}(T)$ be a term of type $Q \multimap Q'$. Then, we can generate a family of circuits $\texttt{C}(t) \triangleq (C_w)_{w\in\texttt{S}(Q)}$ and a polynomial P such that, for any $w \in \texttt{S}(Q)$:*

- *$|C_w| \leq P(T(|w|))$;*
- *$\forall\ v \in \texttt{S}(w)$, C_w approximates v' with probability $\frac{2}{3}$ on input v, where $tv \rightsquigarrow^* v'$.*

Proof. Let $w \in \texttt{S}(Q)$, and $k = |w|$. We produce $\mathcal{C}^t_w$ inductively on the syntax of t, first with no approximation, and show that C^t_w is of size $P(T(k))$, where $P \in \mathbb{N}[X]$. From that, we approximate each gate, using the chosen universal set of gates, up to a precision $1-\epsilon$, such that the overall circuit has a precision $\frac{2}{3}$. This approximation, done on each gate, can be done using Solovay-Kitaev theorem [32], which is known to approximate the original gate to a precision $1-\epsilon$ in time $\mathcal{O}(\log^c(1/\epsilon))$, for a constant c. Choosing $\epsilon = \frac{2}{3P(T(k))}$ satisfies the overall precision [40], and thus, our approximated circuit is of size $P(T(k))\log^c(\frac{3P(T(k))}{2}) = \mathcal{O}(P(T(k))\log^c(P(T(k))))$, which can be bounded by a polynomial $Q \in \mathbb{N}[X]$, evaluated in $P(T(k))$. Therefore, the circuit is of size $Q(P(T(k))) = R(T(k))$ for $R = Q \circ P \in \mathbb{N}[X]$.

Note that due to the fact that $t \in \texttt{CircuitTerms}$, one can always assume that there is no (contr), (equiv) or (shape) rule; by typing, any subterm is also always well-typed. We may also omit w and write $\mathcal{C}^t$.

All obtained polynomials, by induction, are only summations of polynomial, thus there is no exponential blowup on the coefficients. Furthermore, one can check that the introduced ancillary qubits that play a role must be applied onto a gate, thus we have at most $P(T(k))$ ancillary qubits.

- If $t = \texttt{qcase}\, s \left\{|0\rangle \to t_0\,, |1\rangle \to t_1\right\}$, we first put $\mathcal{C}^s_w$. Then, either t_i are values, and this represents an isometry, thus a unitary (up to ancillary

qubits) which will be approximated later; or they are not values, thus of the shape $c(|i\rangle, s_i)$; and we put the circuits $\mathcal{C}^{s_i}_w$, controlled (on $|i\rangle$ for $\mathcal{C}^{s_i}_w$), one after the other, the control wire being the output of $\mathcal{C}^s_w$. As the three circuits have a size bounded by a polynomial, the total size is bounded by a polynomial too.

- If $t = \mathtt{match}_{1\leq i\leq n}\, s\, \{c_i(\overrightarrow{x_i}) \to t_i\}$, as we know that t terminates, and the input has a precise shape, the match will only reduce to a specific branch; thus, we put $\mathcal{C}^s_w$, followed by $\mathcal{C}^{t_i}_w$, for the corresponding branch. It also binds the wires labelled by the variables in $c(\overrightarrow{x_i})$ to the corresponding outputs of t. This yield one swap, between at most $P(T(k))$ wires, thus can be implemented by at most $P(T(k))$ CNOT gates.

- Constructors: 0 generates a classical wire, $S(t)$ adds 1 to the classical wire, $[\,]$ is an empty circuit, and $h :: t, h \times t$ are both circuits put one on top of the other. The bound on the size is obtained directly, as for `qcase`.

- If $t = \lambda x.s$, this will only reroute the wire labelled with x as the first wire of the circuit (i.e., a swap). if x is not used, this wire is traced out. The swap can be implemented, as for `match`, by $P(T(k))$ CNOT gates.

- Suppose $t = \mathtt{letrec}\, f\, x = s$. We generate recursively $\mathcal{C}_s$, which contains boxes for each possible call of f. We do any circuit simplification (e.g. if there is a controlled gate on a wire with value $|0\rangle$, remove the gate), to remove ill-paths. We have $n \leq |t|$ boxes corresponding to f in $\mathcal{C}^s$. However, one can notice that the boxes are controlled on the same wire, by restriction of the syntax of the `qcase`, and are applied on the same wire, also by restriction of `letrec`. This problem is called *branch sequentialization*, and we can transform this into only one gate f, adding $\mathcal{O}(1)$ gates before and after such gate [30, Theorem 9]. Then, we generate $\mathcal{C}^s_{w'}$, replacing the box f in our circuit, but for the new corresponding input shape w'. Such process will terminate, as t terminates, thus there is only a finite number of recursive calls. In the worst case, we have added $T(k)$ circuits, thus the total size is $T(k)P(T(k)) = P'(T(k))$, for $P' = XP \in \mathbb{N}[X]$.

- Other terms are quite direct: $x, |0\rangle, |1\rangle$ generate a wire with this value, st just puts $\mathcal{C}^t$ and then $\mathcal{C}^s_{w'}$, where w' is the shape of t for the specific input w; due to the type restriction, we can assume there is no `shape` construct, else it would just be on natural numbers, which gives the same number back; superpositions are only on values, thus will only create ancillary wires. □

Theorem 21 (Circuit compilation)**.** *Let $t \in \mathtt{CircuitTerms} \cap \mathtt{Time}(T)$ be a term of type $Q \multimap Q'$ for a given $T : \mathbb{N} \to \mathbb{N}$. Then, we can generate a family of circuits $\mathtt{C}(t) \triangleq (C_w)_{w\in\mathtt{S}(Q)}$ such that for any $w \in \mathtt{S}(Q)$ and any $v \in \mathtt{S}(w)$, C_w approximates v' with probability $\frac{2}{3}$ on input v, where $tv \rightsquigarrow^* v'$.*

Proof. The circuit can be generated similarly to the proof of Theorem 25, as `Faithful` only plays a role for the size of the circuit. □

Theorem 27 (`FBQP` soundness)**.** *Let $t \in \mathtt{Hyrql}(\mathrm{Poly})_{\mathtt{QList}}$, and let $f : \{0,1\}^* \to \{0,1\}^*$. If f is computed by $\mathtt{C}(t)$, then $f \in \mathtt{FBQP}$.*

Proof. We need to prove that $(C_n^t)_{n\in\mathbb{N}}$ is a uniform polynomially-sized family of circuits. As $t \in \texttt{Hyrql}(\mathrm{Poly})_{\texttt{QList}}$, there is a polynomial Q such that t terminates in time Q. Therefore, by Theorem 25, the circuit is of size $\mathcal{O}(P(Q(n)))$, thus of polynomial size in n. Note that we have indexed the circuits by n rather than w, but $|w| = n$, as discussed before. The fact that this family is uniform is obtained by building the circuit as done in the proof of Theorem 25: one can build a Turing machine, taking as input the size n (thus the shape), and suppose that it contains intially the encoding of t, thus without the input, onto a tape of the QTM. As the circuit is build inductively on the syntax, and it differs only by the input shape, this creation is uniform, and is done in polynomial time, as the generated circuit is of polynomial size. □

Theorem 28 (FBQP completeness)**.** *Let $f : \{0,1\}^* \to \{0,1\}^* \in \texttt{FBQP}$. Then, there exists $t \in \texttt{Hyrql}(\mathrm{Poly})_{\texttt{QList}}$ such that $\texttt{C}(t)$ computes f.*

Proof. We prove this result by using Yamakami's algebra [53]. This paper defines a function algebra, and then proves that it completely characterize FBQP. Formally, it defines the function class $\square_1^{\mathrm{QP}}$, which is the smallest class of functions including the following gates, for $\theta \in [0, 2\pi) \cap \tilde{\mathbb{C}}$:

$$
\begin{aligned}
I(|\phi\rangle) &\triangleq |\phi\rangle \\
PHASE_\theta(|\phi\rangle) &\triangleq |0\rangle\langle 0|\phi\rangle + e^{i\theta}|1\rangle\langle 1|\phi\rangle \\
ROT_\theta(|\phi\rangle) &\triangleq \cos\theta\,|\phi\rangle + \sin\theta(|1\rangle\langle 0|\phi\rangle - |0\rangle\langle 1|\phi\rangle) \\
NOT(|\phi\rangle) &\triangleq |1\rangle\langle 0|\phi\rangle + |0\rangle\langle 1|\phi\rangle \\
SWAP(|\phi\rangle) &\triangleq \begin{cases} |\phi\rangle & \text{if } l(|\phi\rangle) \leq 1 \\ \sum_{a,b\in\{0,1\}} |ab\rangle\langle ba|\phi\rangle & \text{otherwise} \end{cases}
\end{aligned}
$$

and is closed under the following schemes:

$$
\begin{aligned}
Compo[g,h](|\phi\rangle) &\triangleq g \circ h(|\phi\rangle) \\
Branch[g,h] &\triangleq \begin{cases} |\phi\rangle & \text{if } l(|\phi\rangle) \leq 1 \\ |0\rangle \otimes g(\langle 0|\phi\rangle) + |1\rangle \otimes h(\langle 1|\phi\rangle) & \text{otherwise} \end{cases} \\
kQRec_t[g,h,p|\mathcal{F}_k](|\phi\rangle) &\triangleq \begin{cases} g(|\phi\rangle) & \text{if } l(|\phi\rangle) \leq t \\ h(\sum_{w\in\{0,1\}^k} |w\rangle \otimes f_w(\langle w|p(|\phi\rangle)\rangle)) & \text{otherwise} \end{cases}
\end{aligned}
$$

Here, $|\phi\rangle$ is a quantum state, i.e., belongs to the Hilbert space $\mathbb{C}^{2^n}$ for a given n written in Dirac notation. Its size $l(|\phi\rangle)$ is n. Given $w \in \{0,1\}^n$ with $n \leq l(|\phi\rangle)$, we may write $|\phi\rangle = \sum_i \alpha_i |w_i z_i\rangle$, where $w_i \in \{0,1\}^n$ and $z_i \in \{0,1\}^{l(|\phi\rangle)-n}$; we then abuse the notation by writing $\langle w|\phi\rangle = \sum_i \alpha_i \langle w|w_i\rangle |z_i\rangle$. $\tilde{\mathbb{C}}$ denotes the complex numbers whose real and imaginary parts can both be approximated by a polynomial-time Turing machine. Note that this falls in the restriction that we did in Section 3.2.

This class is proven to be FBQP complete, thus any function of FBQP can be written as a function $\square_1^{\mathrm{QP}}$. We therefore associate any function $\square_1^{\mathrm{QP}}$ with a given term in Hyrql, $[\![\square_1^{\mathrm{QP}}]\!]$, and prove inductively that they belong in $\texttt{Hyrql}(\mathrm{Poly})_{\texttt{QList}}$.

Given a word $w = w_1 \dots w_n \in \{0,1\}^n$, we denote $L(w)$ as the list $|w_1\rangle :: \cdots :: |w_n\rangle :: [\,]$. One can remark that for any function $f \in \square_1^{\mathrm{QP}}$, and any

input $w \in \{0,1\}^*$, $f|w\rangle = v$, where $[\![f]\!]L(w) \rightsquigarrow^* v$, thus they compute the same function. As $[\![f]\!]$ is approximated by the family of circuits $\mathtt{C}([\![f]\!])$ by Theorem 25, then f is also approximated by this family, with precision $\frac{2}{3}$, thus giving us the wanted result.

$$
\begin{aligned}
[\![I]\!] &\triangleq \lambda x.x \\
[\![Ph_\theta]\!] &\triangleq \lambda x.\mathtt{qcase}\, x \left\{|0\rangle \to |0\rangle\ , |1\rangle \to e^{i\theta}|1\rangle\right\} \\
[\![Rot_\theta]\!] &\triangleq \lambda x.\mathtt{qcase}\, x \left\{|0\rangle \to \cos\theta|0\rangle + \sin\theta|1\rangle\ , |1\rangle \to -\sin\theta|0\rangle + \cos\theta|1\rangle\right\} \\
[\![Not]\!] &\triangleq \lambda x.\mathtt{qcase}\, x \left\{|0\rangle \to |1\rangle\ , |1\rangle \to |0\rangle\right\} \\
[\![SWAP]\!] &\triangleq \lambda x.\mathtt{match}\, x \left\{[\,] \to [\,], h :: t \to \mathtt{match}\, t \left\{[\,] \to h :: [\,], h' :: t' \to h' :: h :: t\right\}\right\} \\
[\![COMP[f,g]]\!] &\triangleq \lambda x.([\![f]\!]([\![g]\!]\, x)) \\
[\![Branch[f,g]]\!] &\triangleq \lambda x.\mathtt{match}\, x \left\{\begin{array}{l} [\,] \to [\,] \\ h :: t \to \mathtt{match}\, t \left\{\begin{array}{l} [\,] \to h :: [\,] \\ h' :: t' \to \mathtt{qcase}\, h \left\{\begin{array}{l} |0\rangle \to |0\rangle \otimes [\![f]\!](h' :: t') \\ |1\rangle \to |1\rangle \otimes [\![g]\!](h' :: t') \end{array}\right\} \end{array}\right\} \end{array}\right\}
\end{aligned}
$$

All terms belong in $\mathtt{CircuitTerms} \cap \mathtt{Faithful}$ by construction, and as functions have no recursive construct, they terminate in a fixed time, independently of the input, and thus they belong in $\mathtt{Hyrql}(\mathrm{Poly})_{\mathtt{QList}}$.

For the recursive function, we need to introduce before some constructs. $\mathtt{proj}_t$ is of type $\mathtt{list}(\mathrm{Qbit}) \multimap \mathrm{Qbit}^{\times t} \times \mathtt{list}(\mathrm{Qbit}) \times \mathrm{bit}$; it takes a list x, then returns the first t elements, the remaining part of the list, and a bit indicating if the length of the list is strictly greater than t. If there is less elements, we put $|0\rangle$ by default for the first t elements of the tuple. We then create two functions $\mathtt{check}_t$ which returns the list reconstructed and the bit, and $\tilde{\mathtt{proj}}_t$ which discards the bit. We also define $\mathtt{qcase}_k(x_1,\ldots,x_k)\{|s\rangle \to t_s\}$ as a qcase taking k qubits as inputs and producing all possible 2^k branches, where $s \in \{0,1\}^k$ is the path taken by each x_i; $\overrightarrow{|s\rangle}$ is the notation which transform the k qubits as a list of k elements. This constructs allow then to define the last function of the algebra, with the following set $\mathcal{F}_k = \{f_s : s \in \{0,1\}^k\}$, where each $f_s \in \{kQRec_t[g,h,p|\mathcal{F}_k], I\}$, and $[\![f_s]\!]$ is respectively f or $\lambda x.x$.

$$
\begin{aligned}
\mathtt{proj}_t &\triangleq q\lambda x.\mathtt{match}\, x \left\{\begin{array}{l} [\,] \to (|0\rangle, \ldots, |0\rangle, [\,], 0) \\ h_1 :: r_1 \to \mathtt{match}\, r_1 \ldots \mathtt{match}\, r_t \left\{\begin{array}{c} [\,] \to (h_1,\ldots,h_t,[\,],0) \\ h_{t+1} :: r_{t+1} \to (h_1,\ldots,h_t,h_{t+1} :: r_{t+1},1) \end{array}\right\} \end{array}\right\} \\
\tilde{\mathtt{proj}}_t &\triangleq \lambda x.\mathtt{match}\, \mathtt{proj}_t \left\{(h_1,\ldots,h_d,r,b) \to (h_1,\ldots,h_d,r)\right\} \\
\mathtt{check}_t &\triangleq \lambda x.\mathtt{match}\, \mathtt{proj}_t \left\{(h_1,\ldots,h_d,r,b) \to (h_1 :: \cdots :: h_t :: r, b)\right\}
\end{aligned}
$$

$$
\begin{aligned}
[\![kQrec_t[g,h,p|\mathcal{F}_k]]\!] \triangleq\ & \mathtt{letrec}\, f\, x = \mathtt{match}\, \mathtt{check}_t\, x\{(y,b) \to \mathtt{match}\, b\{0 \to f(y), \\
& 1 \to \mathtt{match}\, \tilde{\mathtt{proj}}_k(h\, y) \left\{(h_1,\ldots,h_k,r) \to g(\mathtt{qcase}_k(h_1,\ldots,h_k))\{|s\rangle \to \overrightarrow{|s\rangle} :: [\![f_s]\!]\, r\}\right\}\}\}
\end{aligned}
$$

Again, such encoding belongs to $\mathtt{CircuitTerms} \cap \mathtt{Faithful}$. Furtheremore, terminating in polynomial time is guaranteed, as:

- By induction, each g, h, p computes in polynomial time;

- Either f_s is the identity, thus stops here, or is a recursive call on an input of size that decreases by k;
- Therefore, the compute time is roughly, for an input of size n, $\frac{n}{k}\max(P_g(n), P_h(p), P_p(n))$, where P_g, P_h, P_p is the compute time of respectively g, h, p, which is polynomial, by induction hypothesis. The obtain time is thus polynomial in n. □

References

[1] Alastair A. Abbott, Julian Wechs, Dominic Horsman, Mehdi Mhalla, and Cyril Branciard. Communication through coherent control of quantum channels. *Quantum*, 4:333, September 2020.

[2] Thorsten Altenkirch and Jonathan Grattage. A functional quantum programming language. In *Logic in Computer Science, LICS'05*, pages 249–258. IEEE Computer Society, 2005.

[3] Mateus Araújo, Fabio Costa, and Časlav Brukner. Computational advantage from quantum-controlled ordering of gates. *Phys. Rev. Lett.*, 113:250402, dec 2014.

[4] Thomas Arts and Jürgen Giesl. Termination of term rewriting using dependency pairs. *Theoretical Computer Science*, 236(1):133–178, 2000.

[5] Martin Avanzini and Georg Moser. Dependency pairs and polynomial path orders. In *International Conference on Rewriting Techniques and Applications, RTA'09*, pages 48–62. Springer, 2009.

[6] Martin Avanzini, Georg Moser, Romain Péchoux, Simon Perdrix, and Vladimir Zamdzhiev. Quantum expectation transformers for cost analysis. In Christel Baier and Dana Fisman, editors, *LICS '22: Symposium on Logic in Computer Science*, pages 10:1–10:13. ACM, 2022.

[7] Patrick Baillot and Ugo Dal Lago. Higher-order interpretations and program complexity. *Information and Computation*, 248:56–81, 2016.

[8] Henk Barendregt. The lambda calculus: its syntax and semantics. *Studies in logic and the foundations of Mathematics*, 1984.

[9] Kathleen Barsse, Romain Péchoux, and Simon Perdrix. A quantum programming language for coherent control, 2025.

[10] Charles H. Bennett and Gilles Brassard. Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science*, 560:7–11, 2014.

[11] Ethan Bernstein and Umesh Vazirani. Quantum Complexity Theory. *SIAM Journal on Computing*, 26(5):1411–1473, October 1997.

[12] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. Silq: a high-level quantum language with safe uncomputation and intuitive semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '20, page 286–300. ACM, June 2020.

[13] Guillaume Bonfante, Jean-Yves Marion, and Jean-Yves Moyen. Quasi-interpretations a way to control resources. *Theoretical Computer Science*, 412(25):2776–2796, June 2011.

[14] Kostia Chardonnet, Louis Lemonnier, and Benoît Valiron. Semantics for a Turing-Complete Reversible Programming Language with Inductive Types. In Jakob Rehof, editor, *Formal Structures for Computation and Deduction (FSCD 2024)*, volume 299 of *LIPIcs*, pages 19:1–19:19, 2024.

[15] Kostia Chardonnet, Alexis Saurin, and Benoît Valiron. A Curry-Howard Correspondence for Linear, Reversible Computation. *LIPIcs, Volume 252, CSL 2023*, 252:13:1–13:18, 2023.

[16] Giulio Chiribella, Giacomo Mauro D'Ariano, Paolo Perinotti, and Benoit Valiron. Quantum computations without definite causal structure. *Phys. Rev. A*, 88:022318, Aug 2013.

[17] Andrea Colledan and Ugo Dal Lago. Flexible type-based resource estimation in quantum circuit description languages. *Proc. ACM Program. Lang.*, 9(POPL), January 2025.

[18] Andrew W. Cross, Lev S. Bishop, John A. Smolin, and Jay M. Gambetta. Open quantum assembly language, 2017.

[19] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, Niel De Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3(3):1–50, September 2022.

[20] Ugo Dal Lago, Andrea Masini, and Margherita Zorzi. Quantum implicit computational complexity. *Theor. Comput. Sci.*, 411(2):377–409, 2010.

[21] Kinnari Dave, Louis Lemonnier, Romain Péchoux, and Vladimir Zamdzhiev. Combining quantum and classical control: syntax, semantics and adequacy. In *Foundations of Software Science and Computation Structures, FoSSaCS 2025*, pages 155–175, Berlin, Heidelberg, 2025. Springer-Verlag.

[22] Nachum Dershowitz. Orderings for term-rewriting systems. *Theoretical computer science*, 17(3):279–301, 1982.

[23] Nachum Dershowitz. Termination of rewriting. *J. Symb. Comput.*, 3(1-2):69–115, February 1987.

[24] Alejandro Díaz-Caro and Octavio Malherbe. Quantum control in the unitary sphere: Lambda-s1 and its categorical model. *Log. Methods Comput. Sci.*, 18(3), 2022.

[25] Jörg Endrullis, Herman Geuvers, and Hans Zantema. *Degrees of Undecidability in Term Rewriting*, pages 255–270. Springer Berlin Heidelberg, 2009.

[26] Yuan Feng and Mingsheng Ying. Quantum Hoare logic with classical variables. *ACM Transactions on Quantum Computing*, 2(4):1–43, 2021.

[27] Florent Ferrari, Emmanuel Hainry, Romain Péchoux, and Mário Silva. Quantum programming in polylogarithmic time. In *Mathematical Foundations of Computer Science, MFCS 2025*, volume 345 of *LIPIcs*, pages 47:1–47:17, 2025.

[28] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. Quipper: a scalable quantum programming language. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13*, pages 333–342. ACM, June 2013.

[29] Emmanuel Hainry, Romain Péchoux, and Mário Silva. A programming language characterizing quantum polynomial time. In *Foundations of Software Science and Computation Structures, Fossacs 2023*, pages 156–175. Springer, April 2023.

[30] Emmanuel Hainry, Romain Péchoux, and Mário Silva. Branch sequentialization in quantum polytime. In Maribel Fernández, editor, *Formal Structures for Computation and Deduction, FSCD 2025*, volume 337 of *LIPIcs*, pages 22:1–22:22, 2025.

[31] Vesa Halava, Tero Harju, Mika Hirvensalo, and Juhani Karhumäki. Skolem's problem - on the border between decidability and undecidability. Technical Report 683, Turku Center for Computer Science, 2005.

[32] Alexei Y. Kitaev. Quantum computations: algorithms and error correction. *Russian Mathematical Surveys*, 52(6):1191–1249, December 1997.

[33] Emanuel Knill. Conventions for quantum pseudocode. Technical report, Los Alamos National Lab., 1996.

[34] Hlér Kristjánsson, Tatsuki Odake, Satoshi Yoshida, Philip Taranto, Jessica Bavaresco, Marco Túlio Quintino, and Mio Murao. Exponential separation in quantum query complexity of the quantum switch with respect to simulations with standard quantum circuits, 2024.

[35] Yves Lafont. Soft linear logic and polynomial time. *Theor. Comput. Sci.*, 318(1-2):163–180, 2004.

[36] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The size-change principle for program termination. In *Symposium on Principles of Programming Languages, POPL 2001*, pages 81–92, New York, NY, USA, January 2001. ACM.

[37] Louis Lemonnier. *The Semantics of Effects : Centrality, Quantum Control and Reversible Recursion*. PhD thesis, Université Paris-Saclay, June 2024.

[38] Jean-Yves Marion and Romain Péchoux. Sup-interpretations, a semantic method for static analysis of program resources. *ACM Trans. Comput. Logic*, 10(4), August 2009.

[39] Georg Moser. *Proof Theory at Work: Complexity Analysis of Term Rewrite Systems*. Cumulative habilitation thesis, University of Innsbruck, 2009.

[40] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, June 2012.

[41] André Nies. *Computability and Randomness*. Oxford University Press, 01 2009.

[42] Romain Péchoux. *Complexité implicite : bilan et perspectives*. Accreditation to supervise research, Université de Lorraine, October 2020.

[43] Lorenzo M. Procopio, Amir Moqanaki, Mateus Araújo, Fabio Costa, Irati Alonso Calafell, Emma G. Dowd, Deny R. Hamel, Lee A. Rozema, Časlav Brukner, and Philip Walther. Experimental superposition of orders of quantum gates. *Nature Communications*, 6(1), August 2015.

[44] Amr Sabry, Benoît Valiron, and Juliana Kaizer Vizzotto. From symmetric pattern-matching to quantum control. In Christel Baier and Ugo Dal Lago, editors, *Foundations of Software Science and Computation Structures*, pages 348–364, Cham, 2018. Springer International Publishing.

[45] Peter Selinger. Towards a quantum programming language. *Mathematical Structures in Computer Science*, 14(4):527–586, August 2004.

[46] Peter Selinger and Benoît Valiron. A lambda calculus for quantum computation with classical control. In Paweł Urzyczyn, editor, *Typed Lambda Calculi and Applications, TLCA 2005*, pages 354–368. Springer Berlin Heidelberg, 2005.

[47] Peter Selinger and Benoît Valiron. Semantic techniques in quantum computation. In Simon Gay and Ian Mackie, editors, *Quantum lambda calculus*, pages 135–172. Cambridge University Press, 2009.

[48] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[49] Márcio M. Taddei, Jaime Cariñe, Daniel Martínez, Tania García, Nayda Guerrero, Alastair A. Abbott, Mateus Araújo, Cyril Branciard, Esteban S. Gómez, Stephen P. Walborn, Leandro Aolita, and Gustavo Lima. Computational advantage from the quantum superposition of multiple temporal orders of photonic gates. *PRX Quantum*, 2:010320, Feb 2021.

[50] Peter van EMDE BOAS. *Machine Models and Simulations*, page 1–66. Handbook of Theoretical Computer Science. Elsevier, 1990.

[51] Finn Voichick, Liyi Li, Robert Rand, and Michael Hicks. Qunity: A unified language for quantum and classical computing. *Proc. ACM Program. Lang.*, 7(POPL), January 2023.

[52] Toshiyuki Yamada. Confluence and Termination of Simply Typed Term Rewriting Systems. In Aart Middeldorp, editor, *Rewriting Techniques and Applications*, pages 338–352, Berlin, Heidelberg, 2001. Springer.

[53] Tomoyuki Yamakami. A Schematic Definition of Quantum Polynomial Time Computability. *The Journal of Symbolic Logic*, 85(4):1546–1587, December 2020.

[54] Mingsheng Ying. *Foundations of quantum programming*. Elsevier, 2024.

[55] Mingsheng Ying and Zhicheng Zhang. Verification of recursively defined quantum circuits, 2024.

[56] Charles Yuan, Agnes Villanyi, and Michael Carbin. Quantum control machine: The limits of control flow in quantum programming. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):1–28, April 2024.